Robot Learning: From Fundamentals to Foundation Models

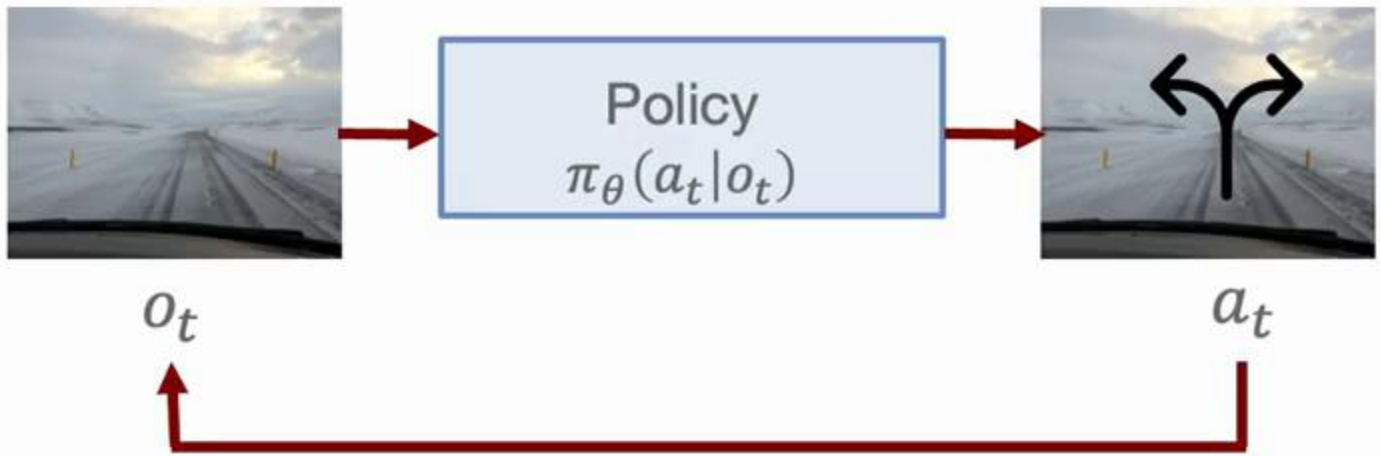
Lecture 7: Sequence Modeling & Transformers

Oier Mees

ETH zürich Microsoft

30.03.2026

Recap: Limitations of Reactive Policies



One Snapshot Can't Model Full State

Memory: what did I do 10 seconds ago? How fast are pedestrians moving?
Action Smoothness: single step model might produce jerky movements

Robotics as a Sequence Modeling Problem

- Robots usually operate in a POMDP, which require reasoning over history $\tau = (o_0, a_0, o_1, a_1, \ldots, o_T)$

$$p_\pi(\tau) = \rho(s_0) \prod_{t=0}^{T-1} \pi(a_t|o_t) P(o_{t+1}|o_t, a_t)$$

Trajectory is a sequence, but the policy is single-step and reactive

Autoregressive Models

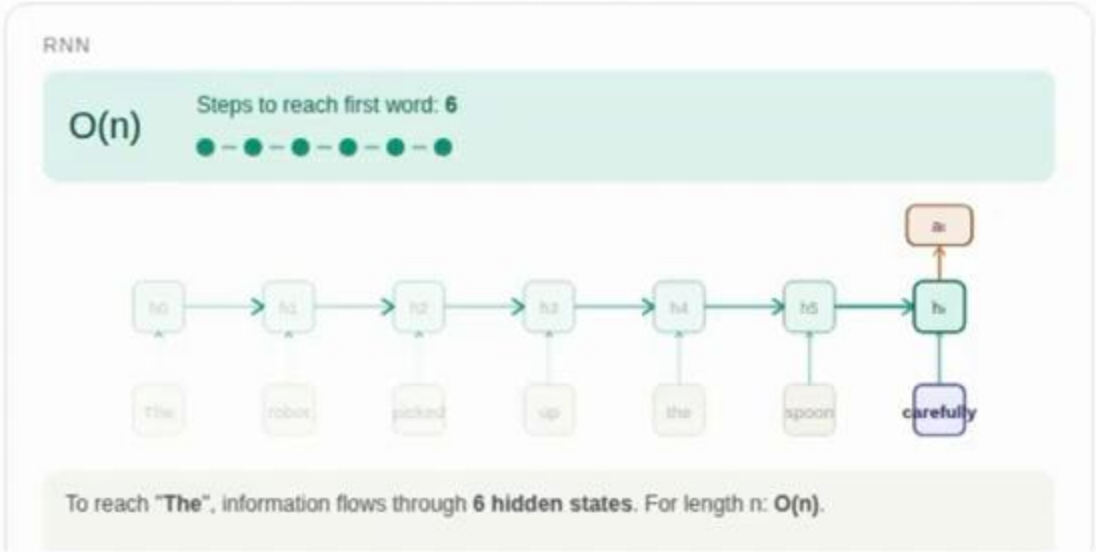
- Any joint distribution can be written as a product of conditionals (chain rule of probability)

$$p_\theta(x) = \prod_{i=1}^{T} p_\theta(x_i \mid x_{1:i-1})$$

How do we learn each conditional?

Modeling Sequences with RNNs

- SOTA until 2016 in NLP, process sequences sequentially
- Limitations:
  - Learn long-range dependencies, exploding/vanishing gradients
  - Parallelize training due to recurrence
  - Retain information, fixed-size h_t state compresses all history



$$h_t = f(h_{t-1}, x_t)$$

Long Short-Term Memory
Hochreiter & Schmidhuber (1997)

Sequence to Sequence Learning with Neural Networks
Sutskever et al., (2014)

Transformer

- Replace recurrence with attention, giving direct $O(1)$, equal access to the entire history

Attention Is All You Need
Vaswani et al., (2017)

Transformer: Attention Mechanism

Q = eWQ, K = eWK, V = eWV

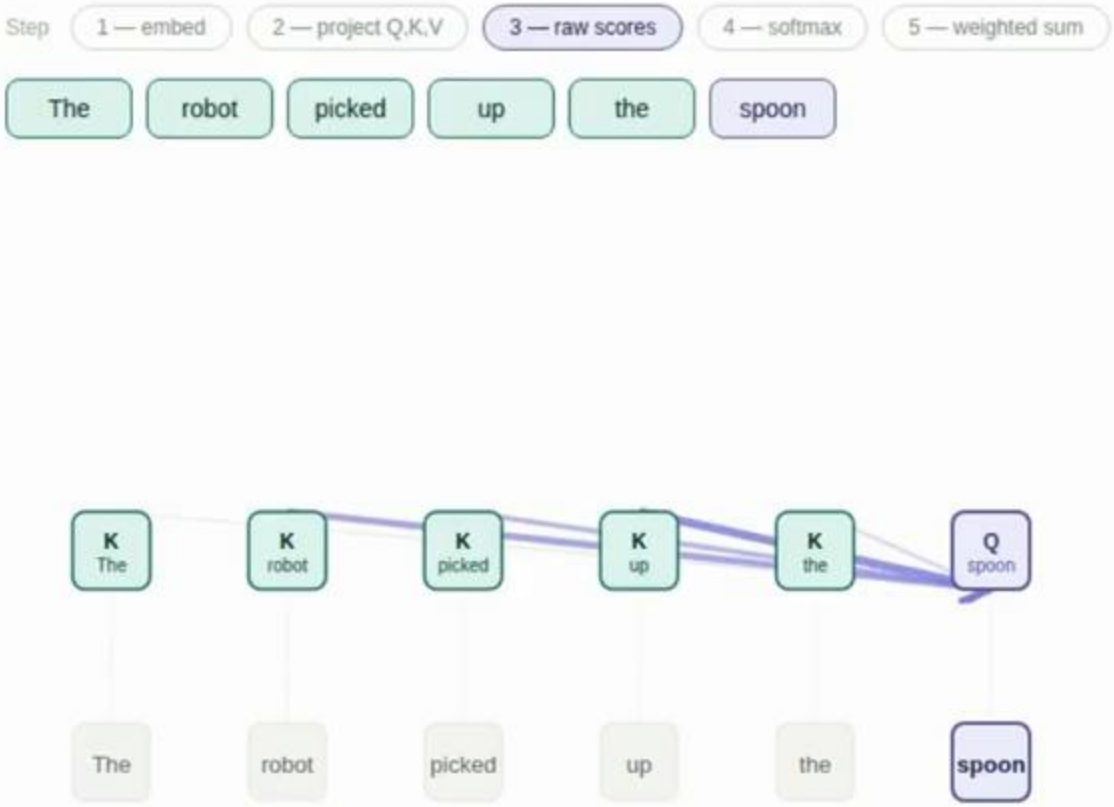
Every token creates three vectors via learned weight matrices: Query (what am I looking for?), Key (what do I advertise to others?), Value (what I will contribute if attended to).

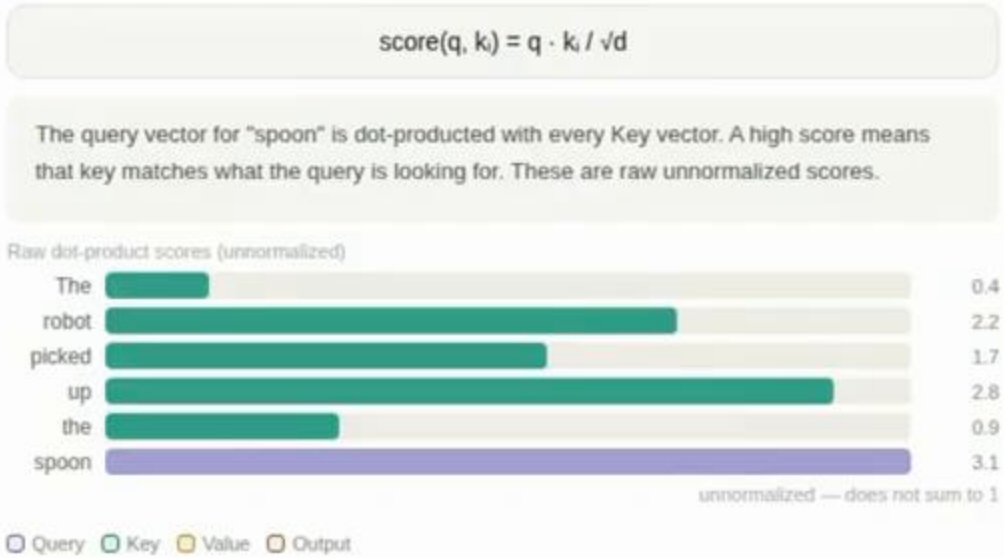
Query Key Value Output

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

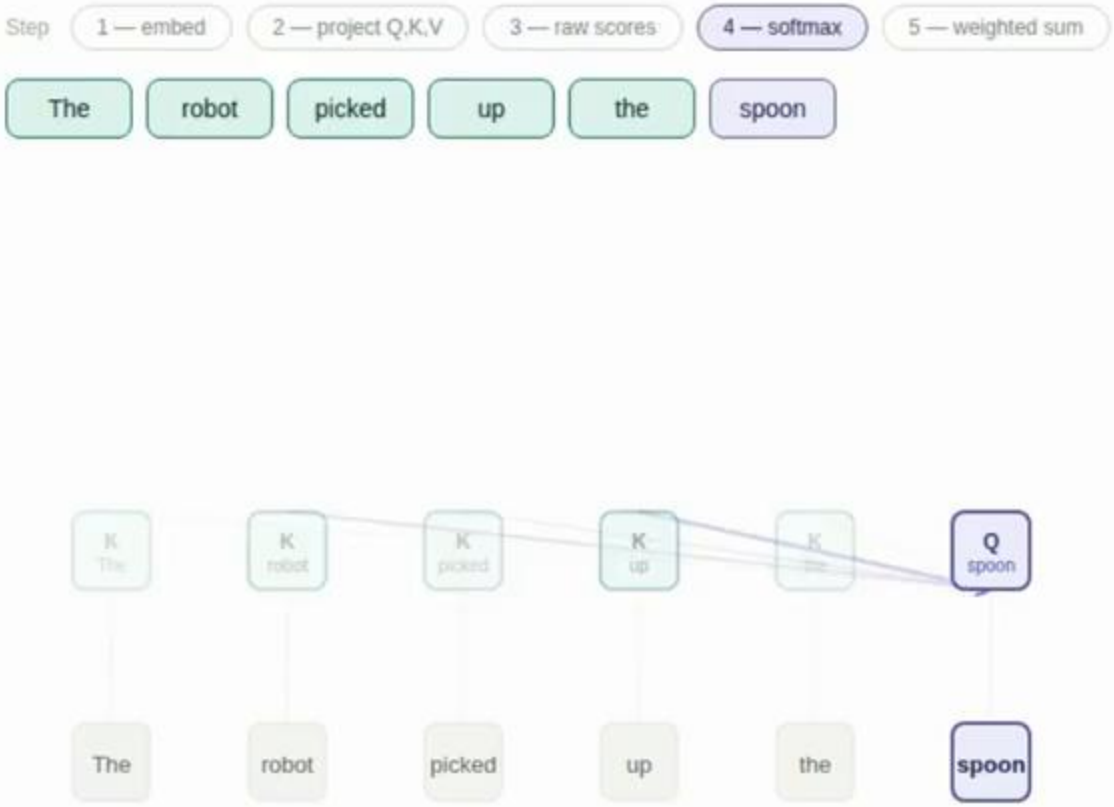


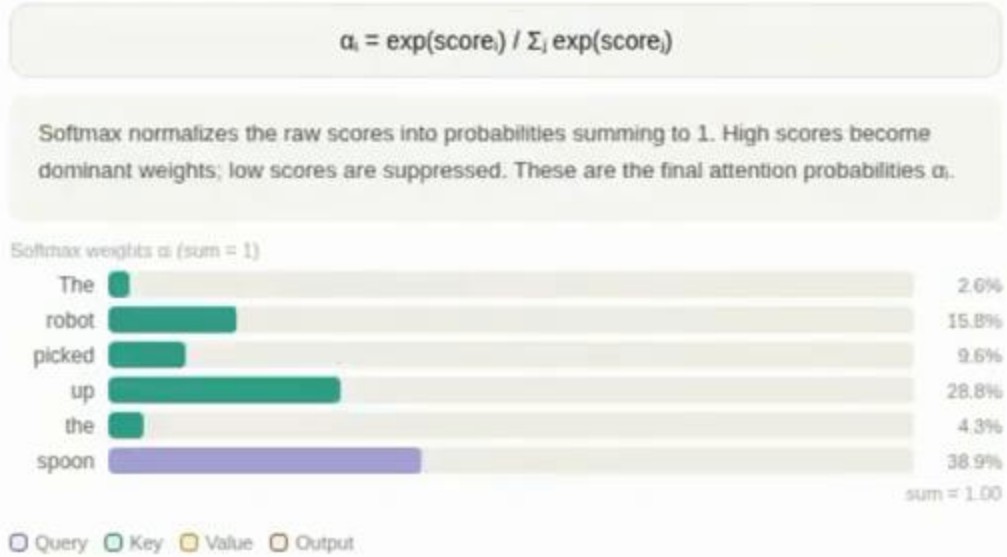


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

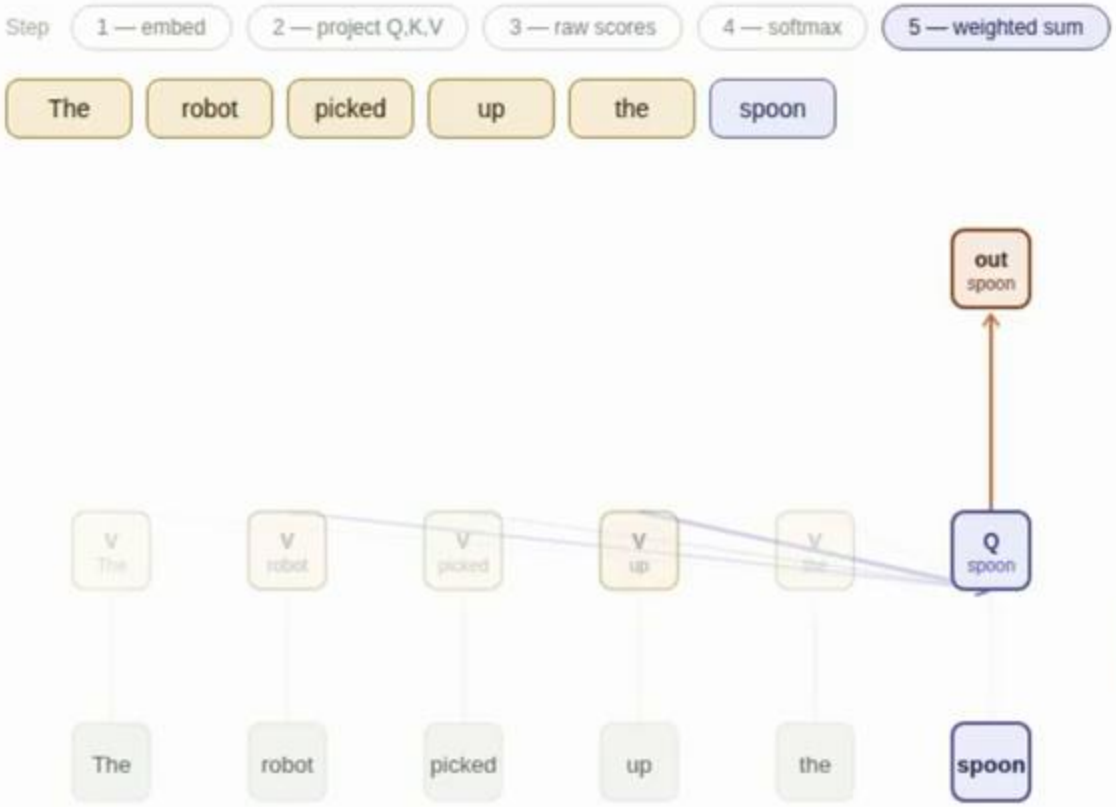




$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

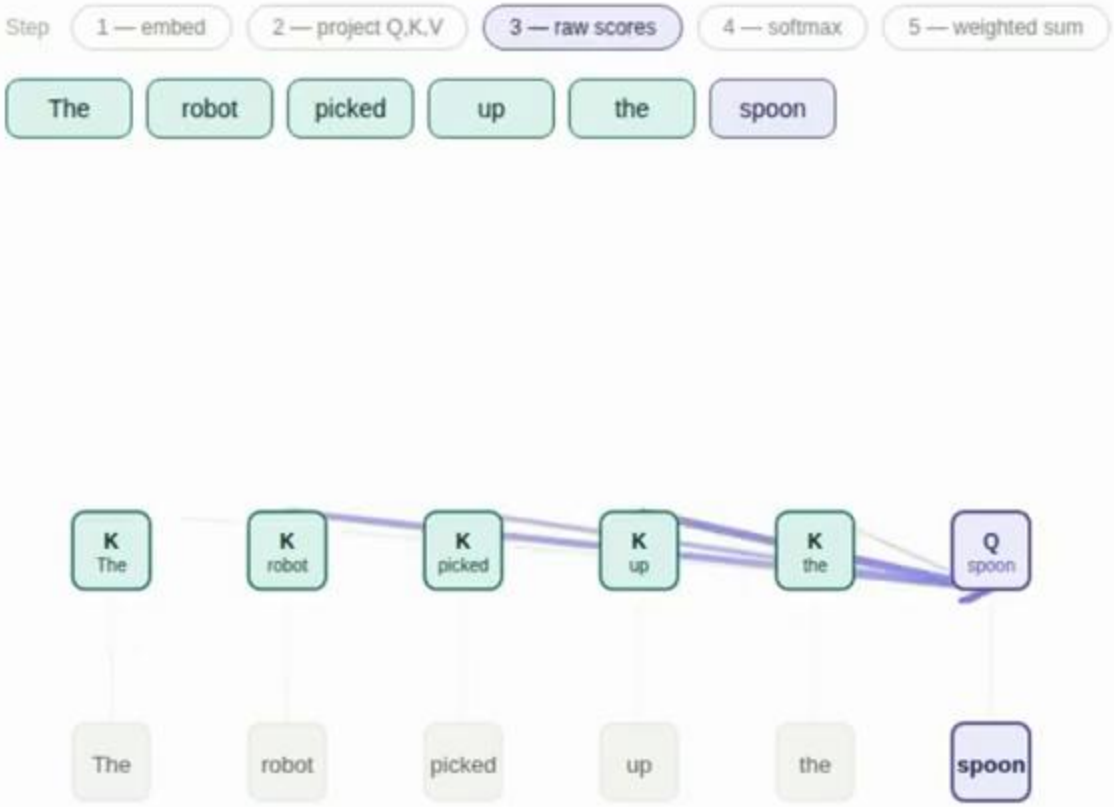
- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

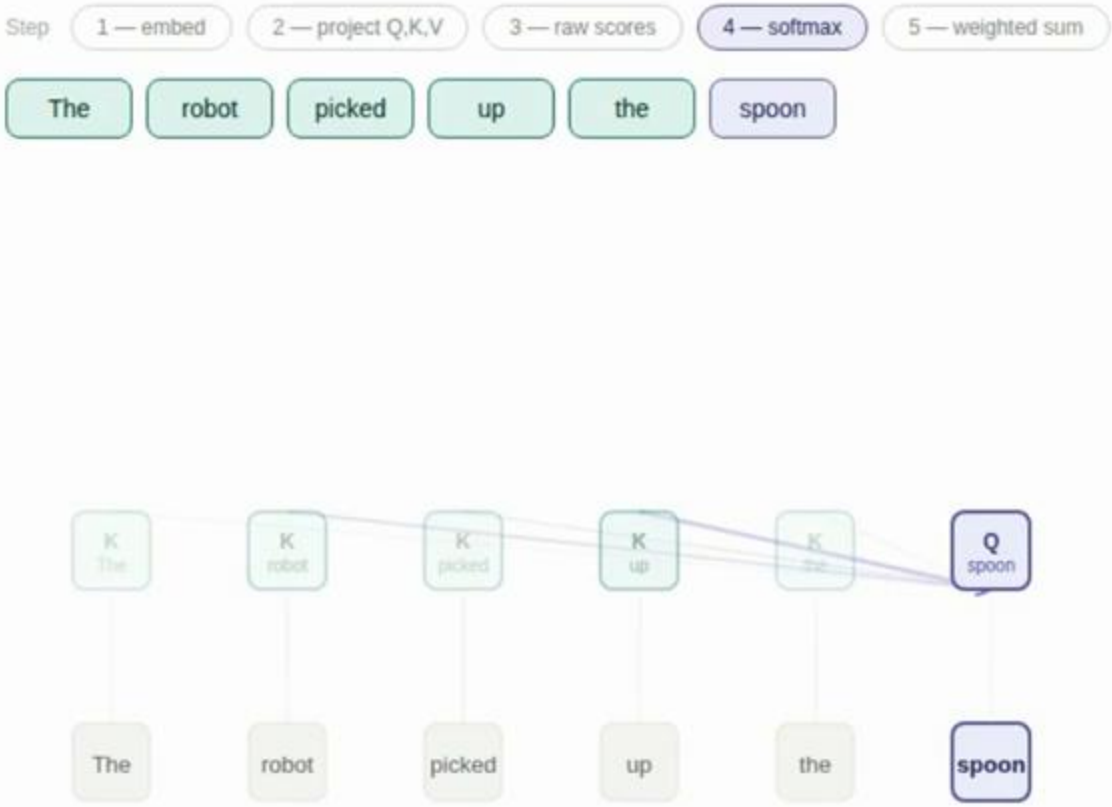
Transformer: Attention Mechanism

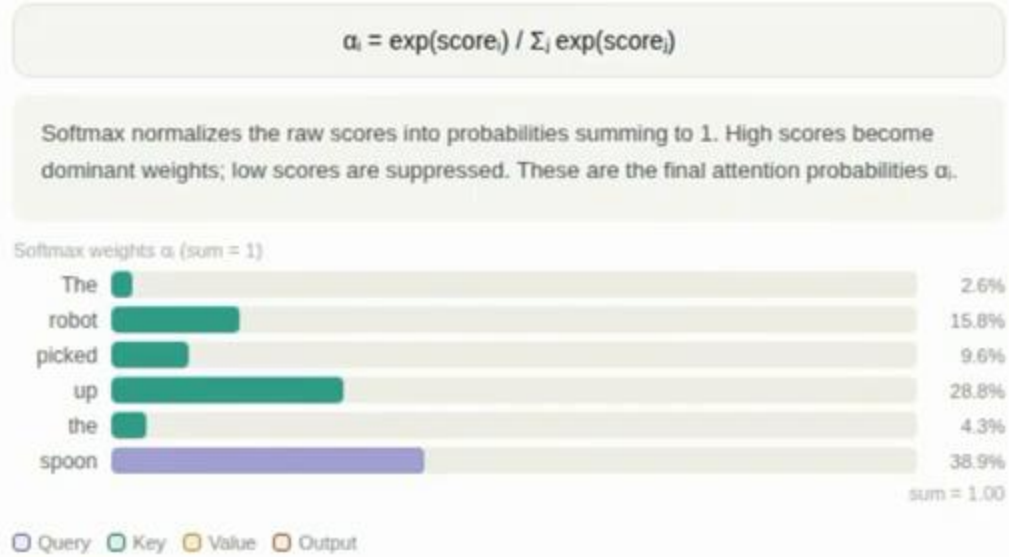


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

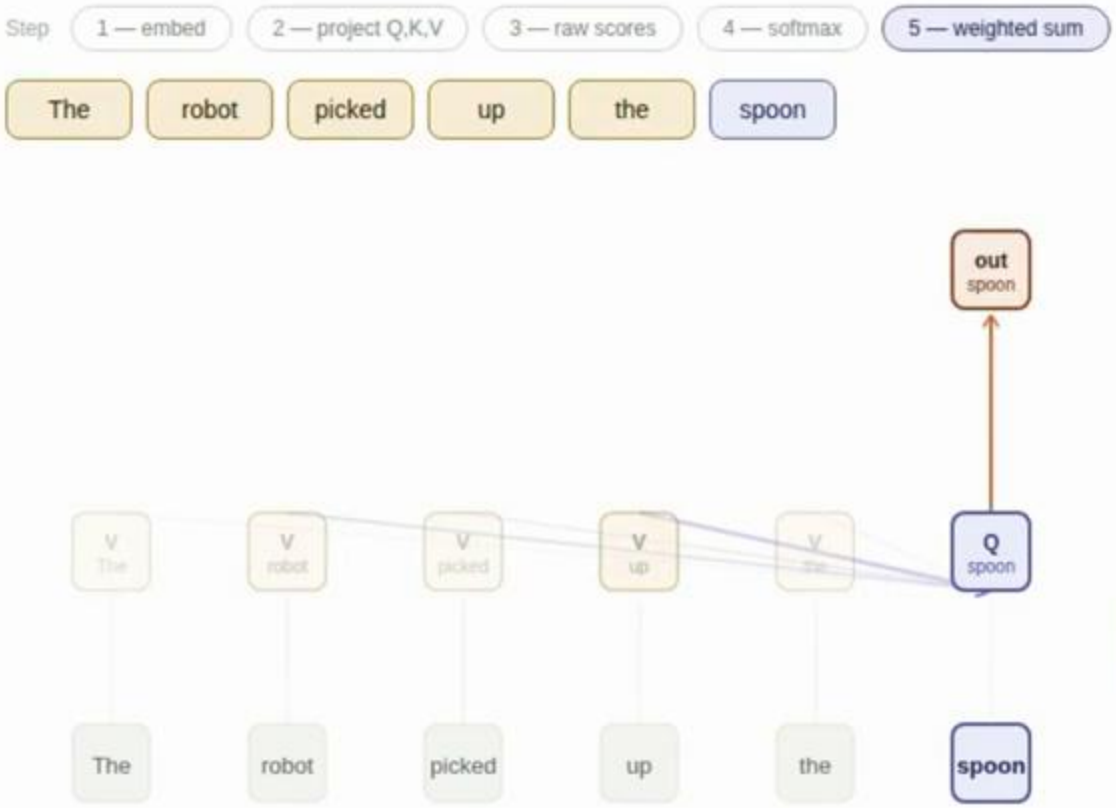




$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Q = eWQ, K = eWK, V = eWV

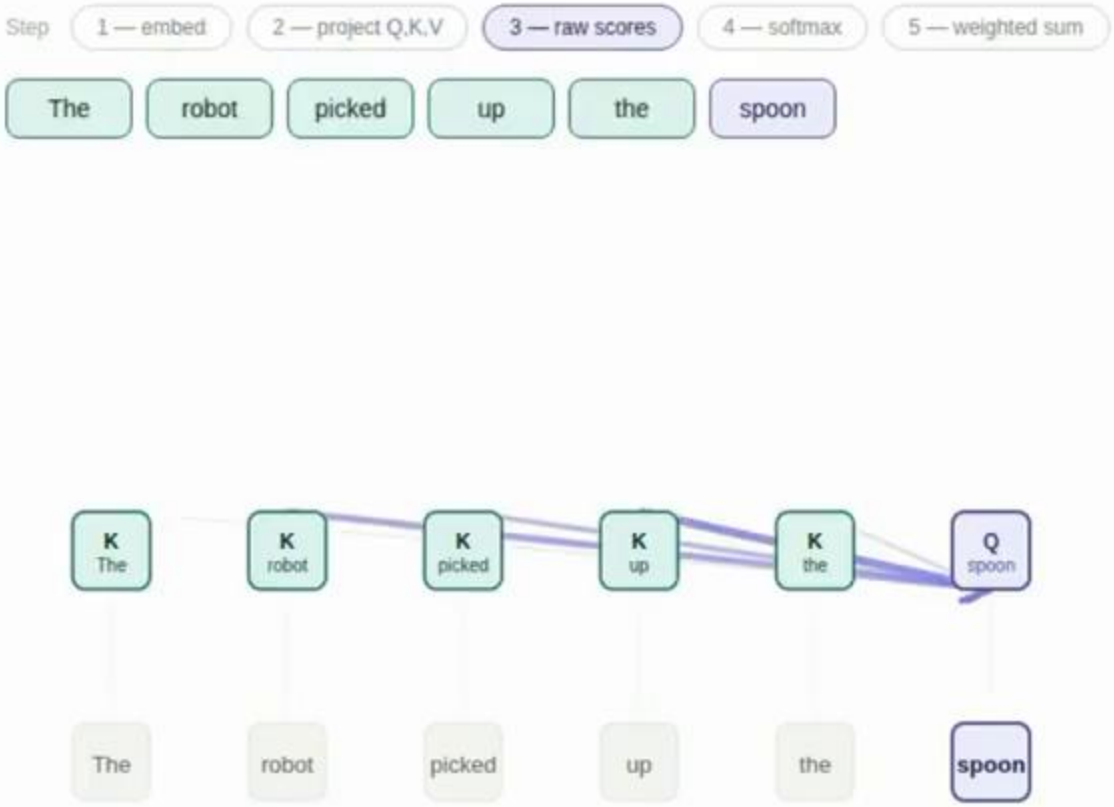
Every token creates three vectors via learned weight matrices: Query (what am I looking for?), Key (what do I advertise to others?), Value (what I will contribute if attended to).

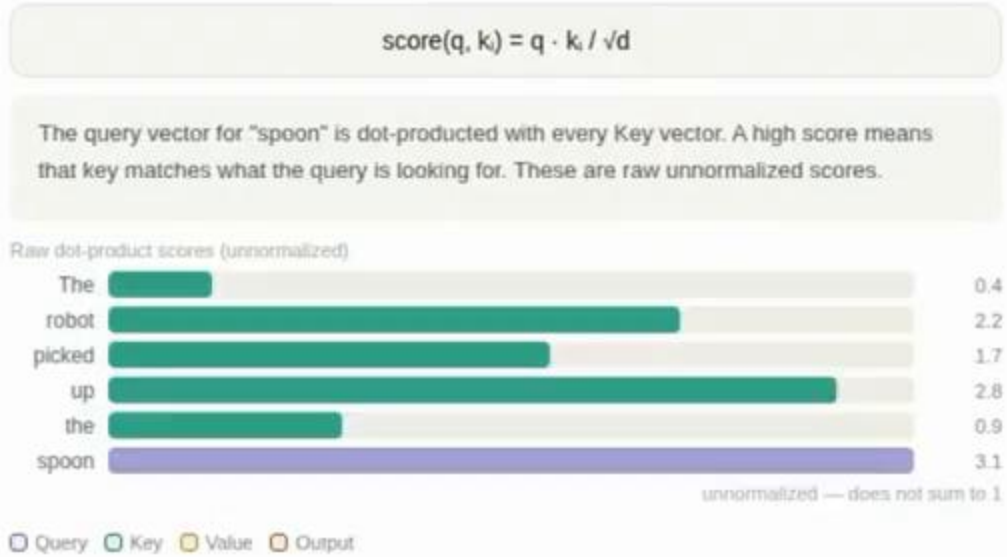
Query Key Value Output

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

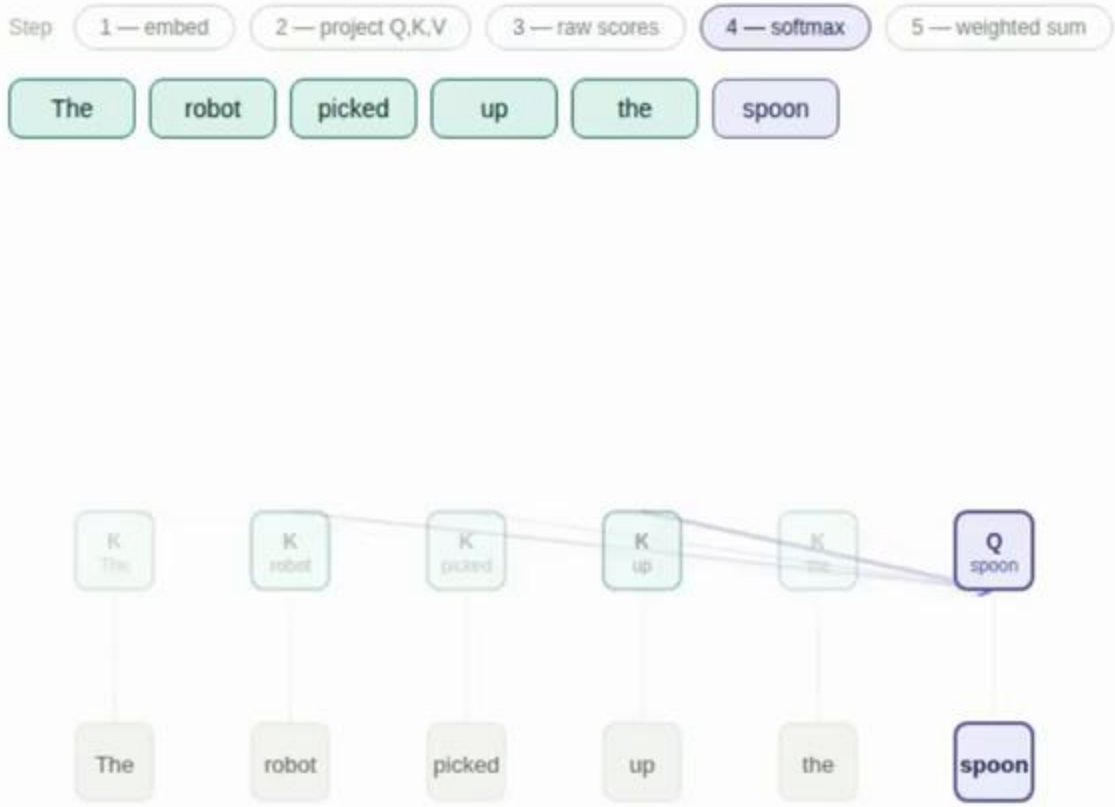




$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

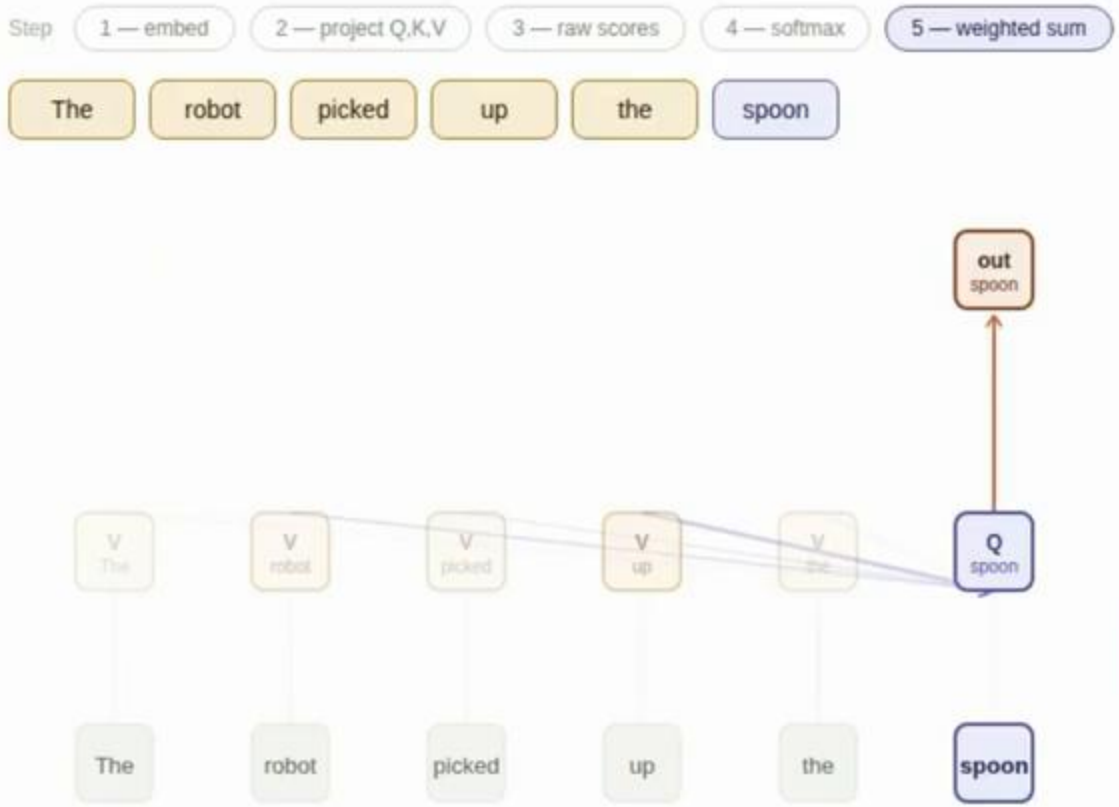
Transformer: Attention Mechanism



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Q = eWQ, K = eWK, V = eWV

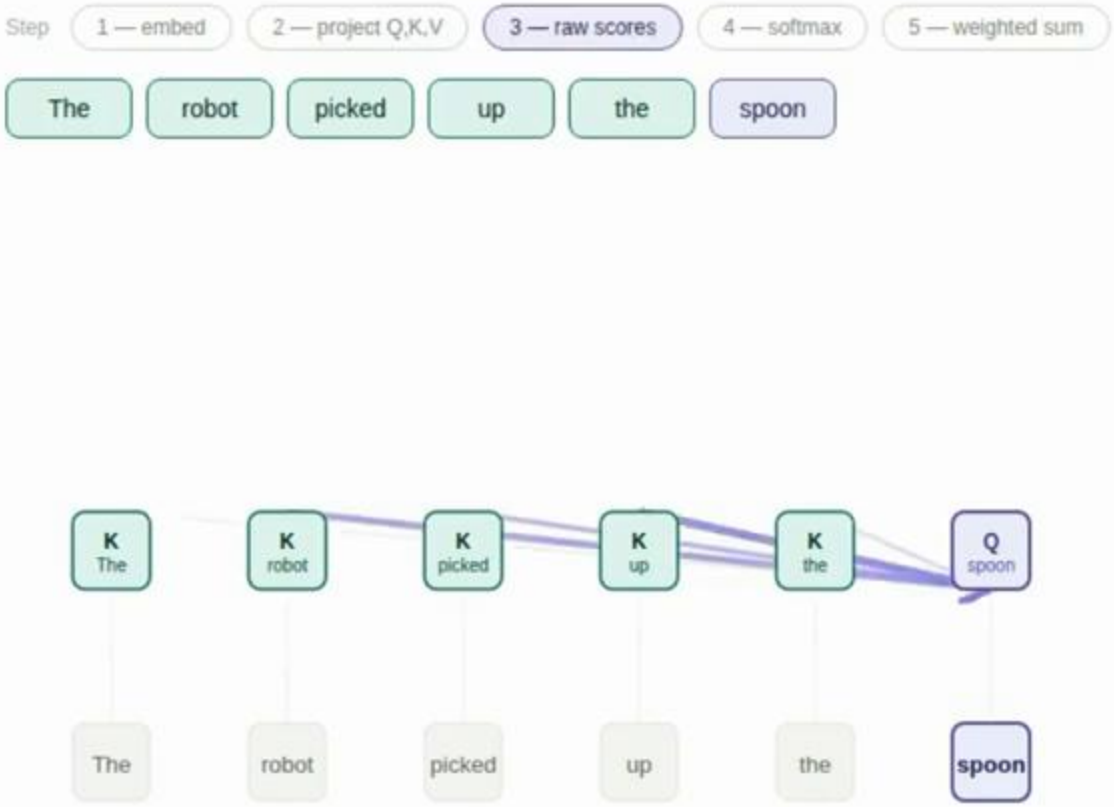
Every token creates three vectors via learned weight matrices: Query (what am I looking for?), Key (what do I advertise to others?), Value (what I will contribute if attended to).

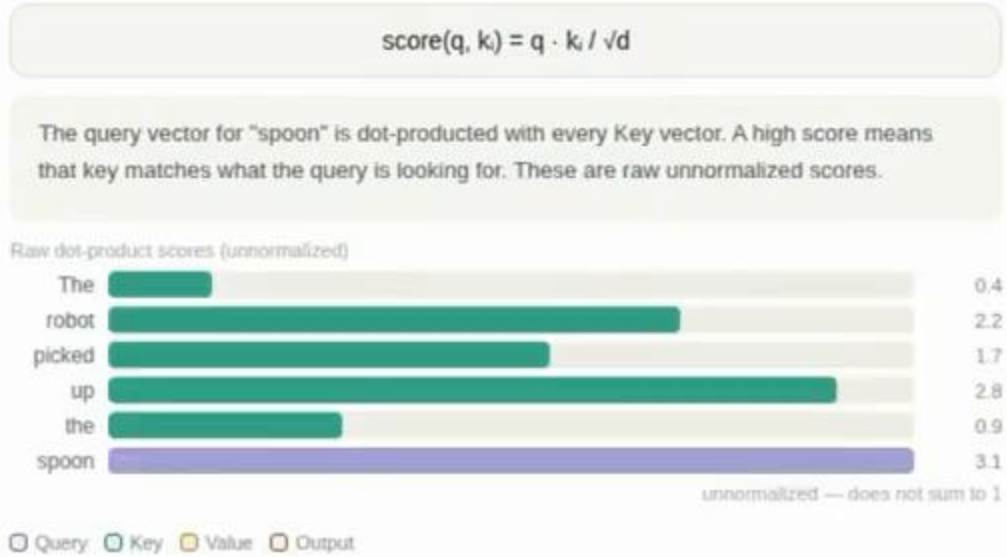
Query Key Value Output

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

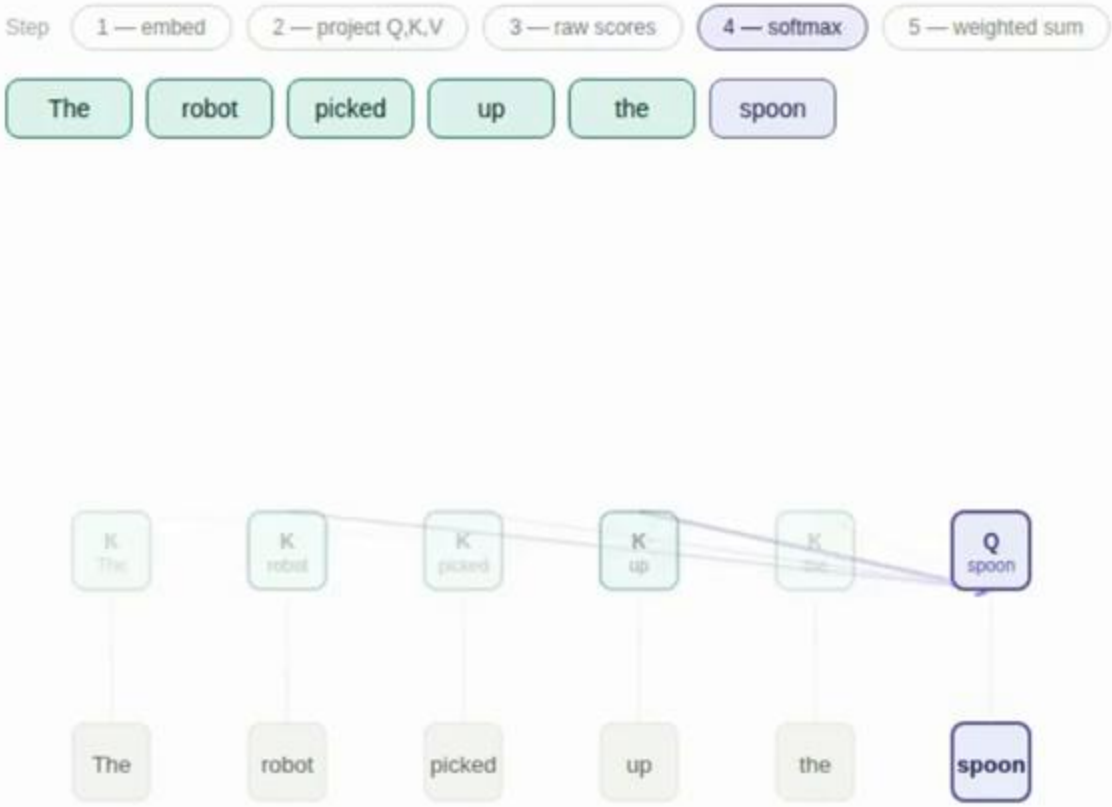


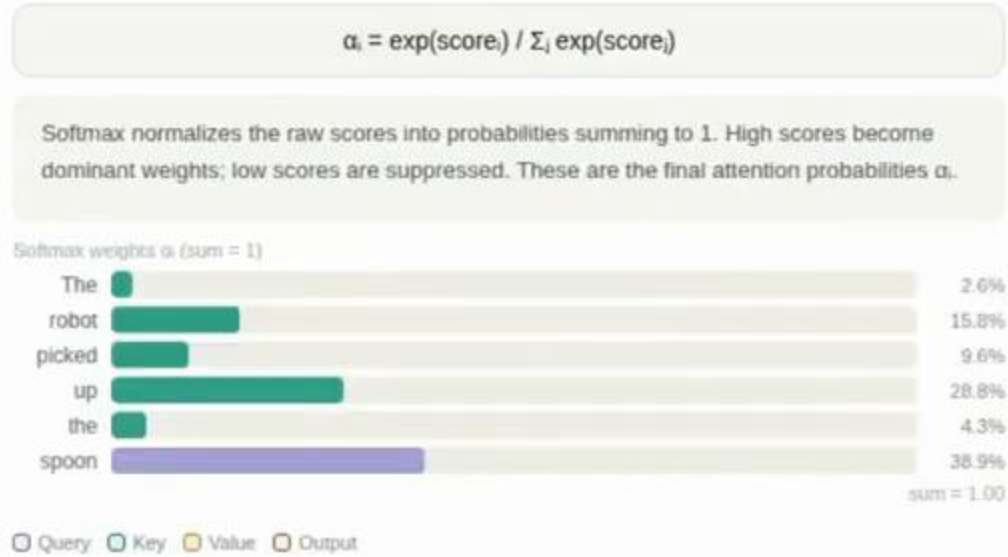


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

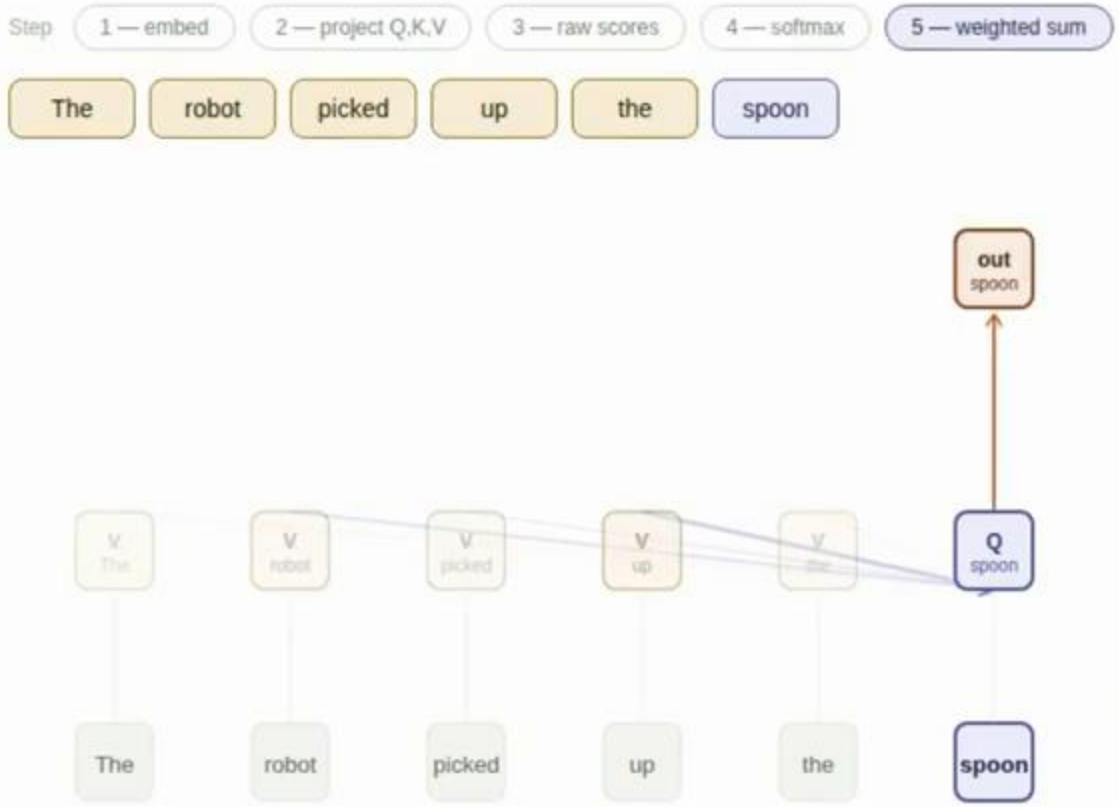




$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

Q = eWQ, K = eWK, V = eWV

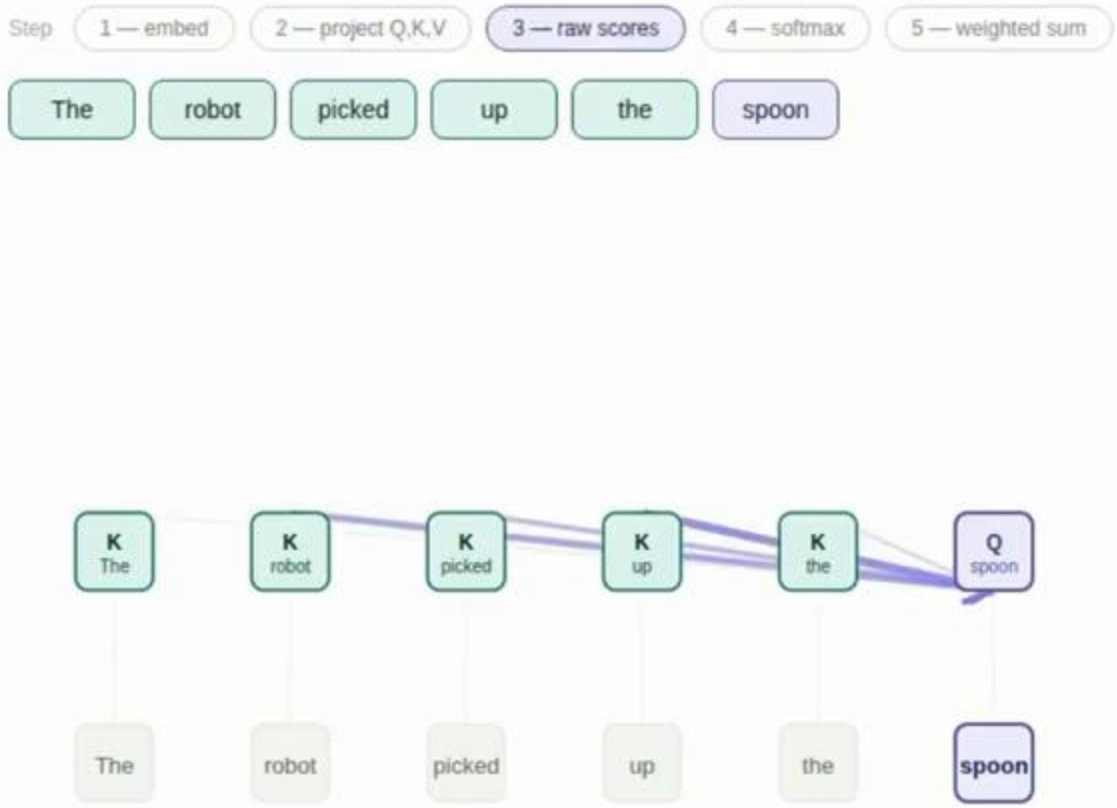
Every token creates three vectors via learned weight matrices: Query (what am I looking for?), Key (what do I advertise to others?), Value (what I will contribute if attended to).

Query Key Value Output

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

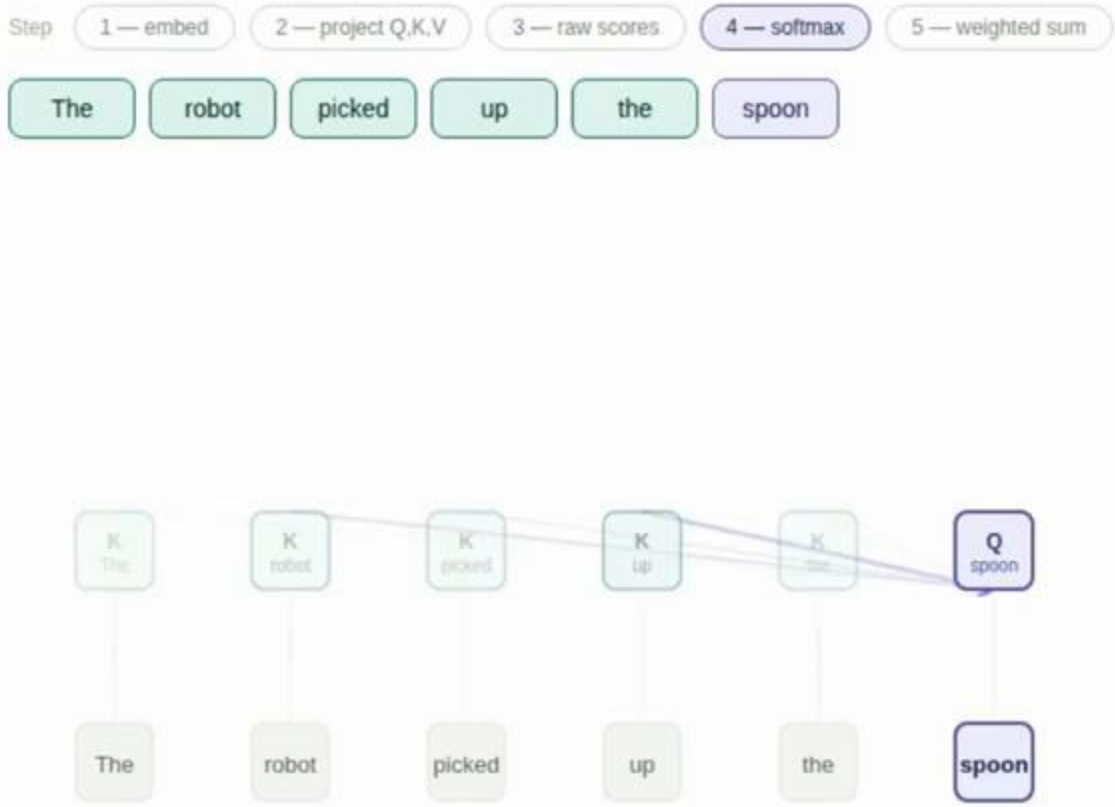
Transformer: Attention Mechanism

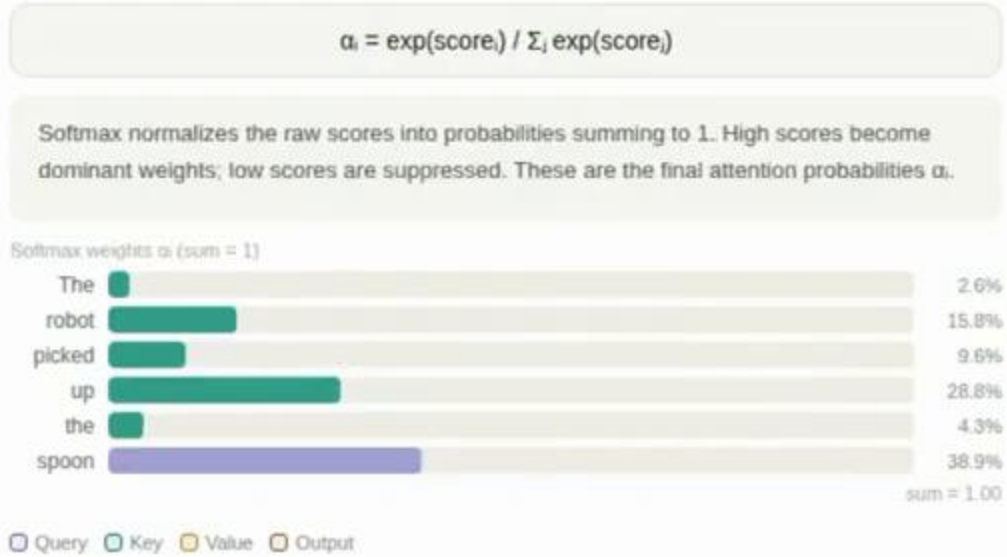


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism

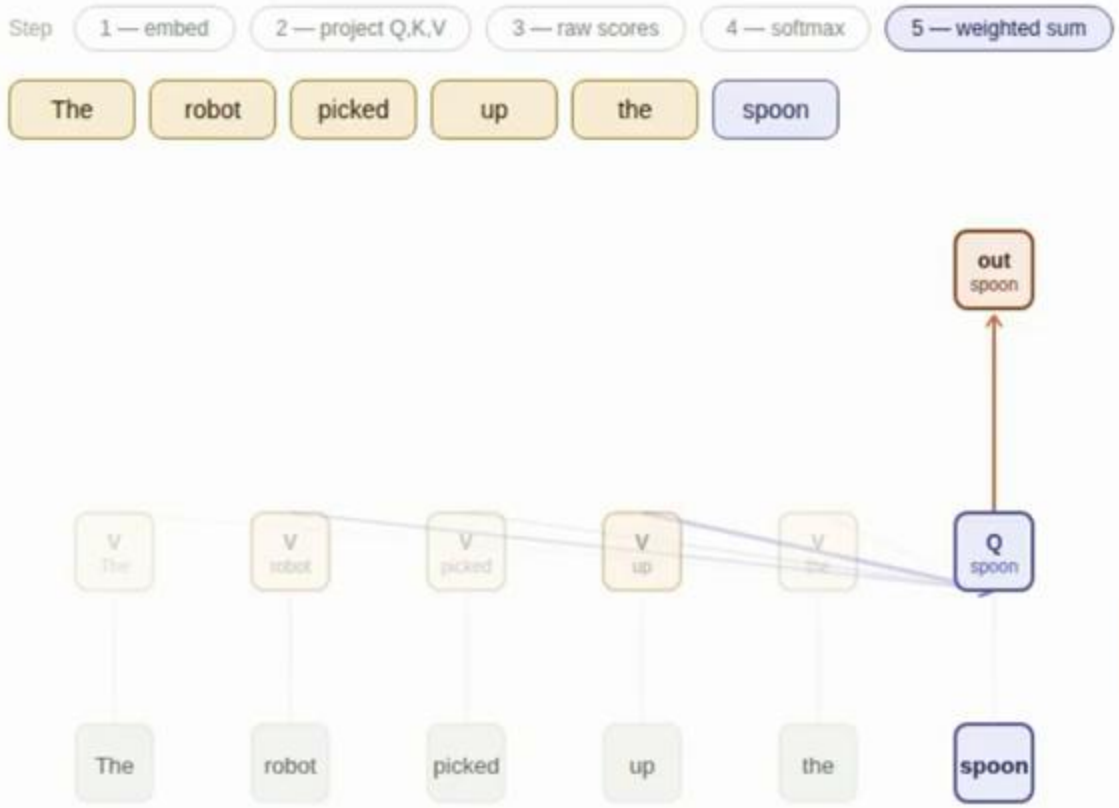




$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Transformer: Attention Mechanism



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

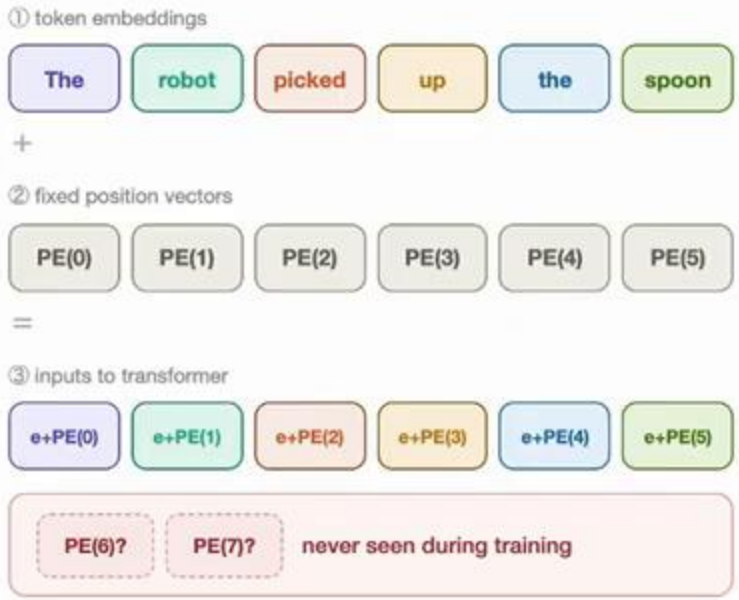
- Compare current hidden state (query) to all past hidden states (keys)
- Construct attention distribution to figure out what parts of the history are relevant via a softmax

Positional Encodings

- Attention has no inherent notion of order
- Inject order information to tokens

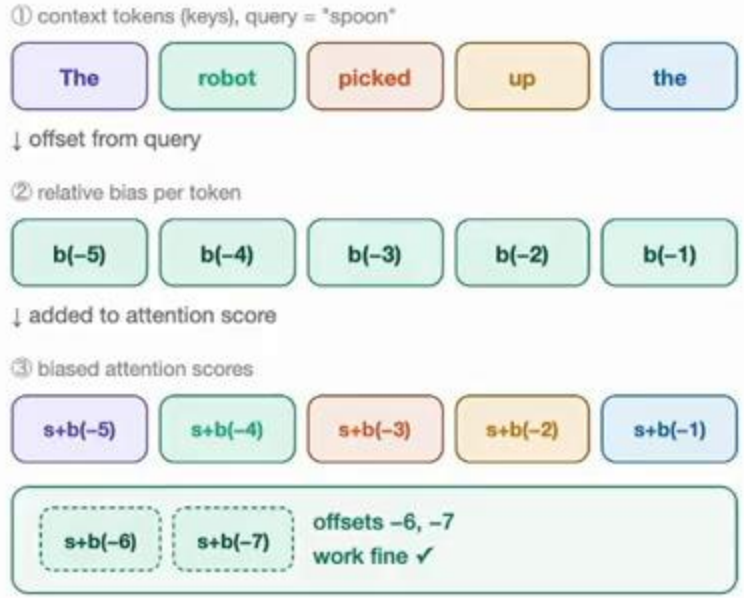
Absolute Positional Encoding

A fixed vector PE is added to each token embedding before the transformer sees it.



Relative Positional Encoding

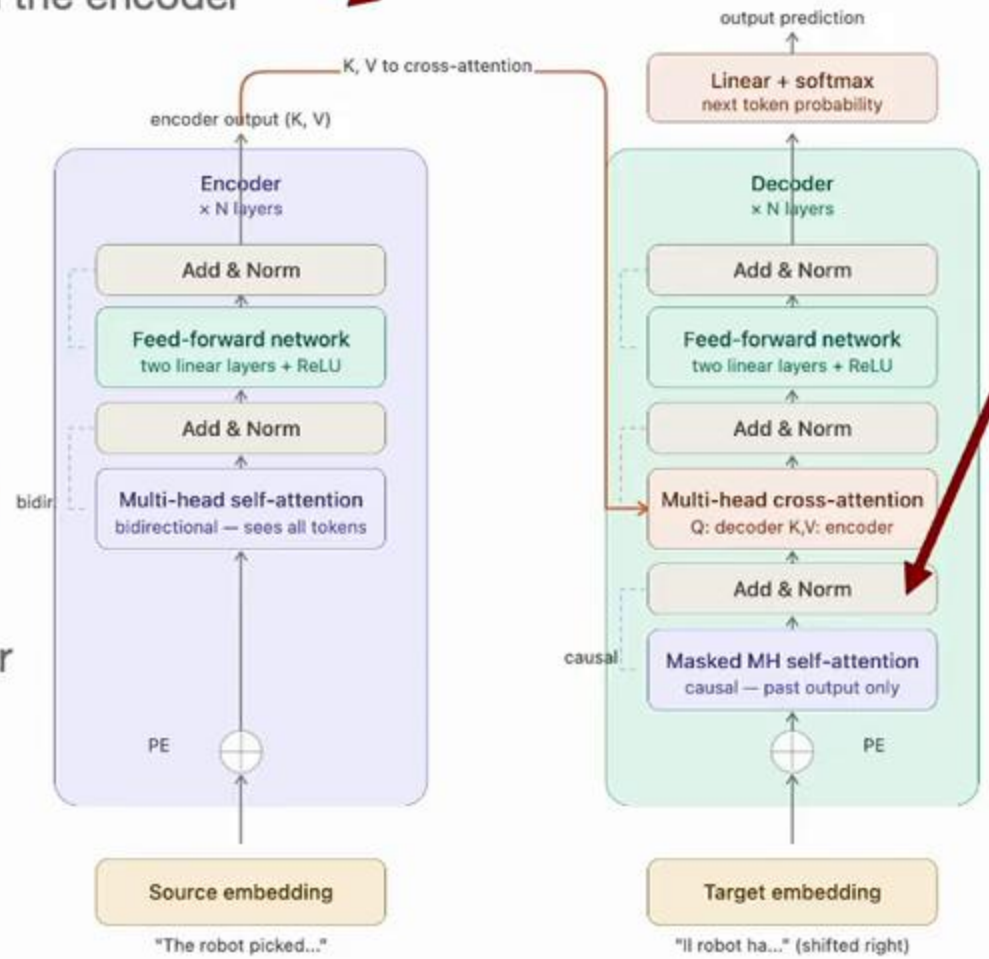
A bias b(i–j) based on the distance between tokens is added to the attention score, not the embedding.



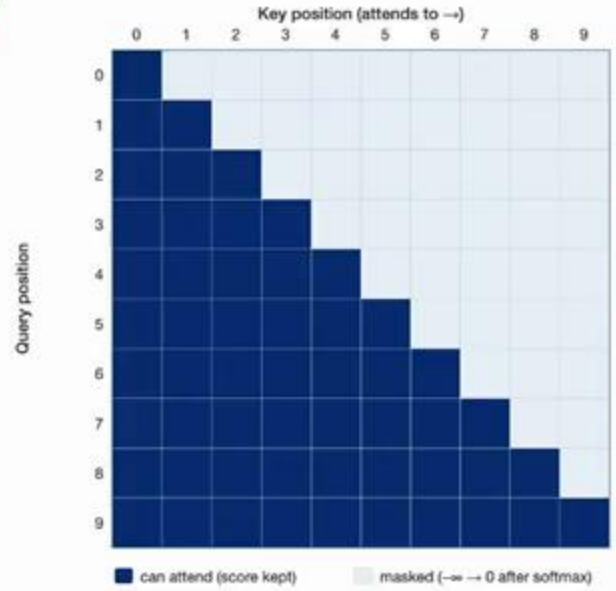
Original Transformer Architecture

Cross-attention
Same as self-attention, but Q comes from the decoder, K and V come from the encoder

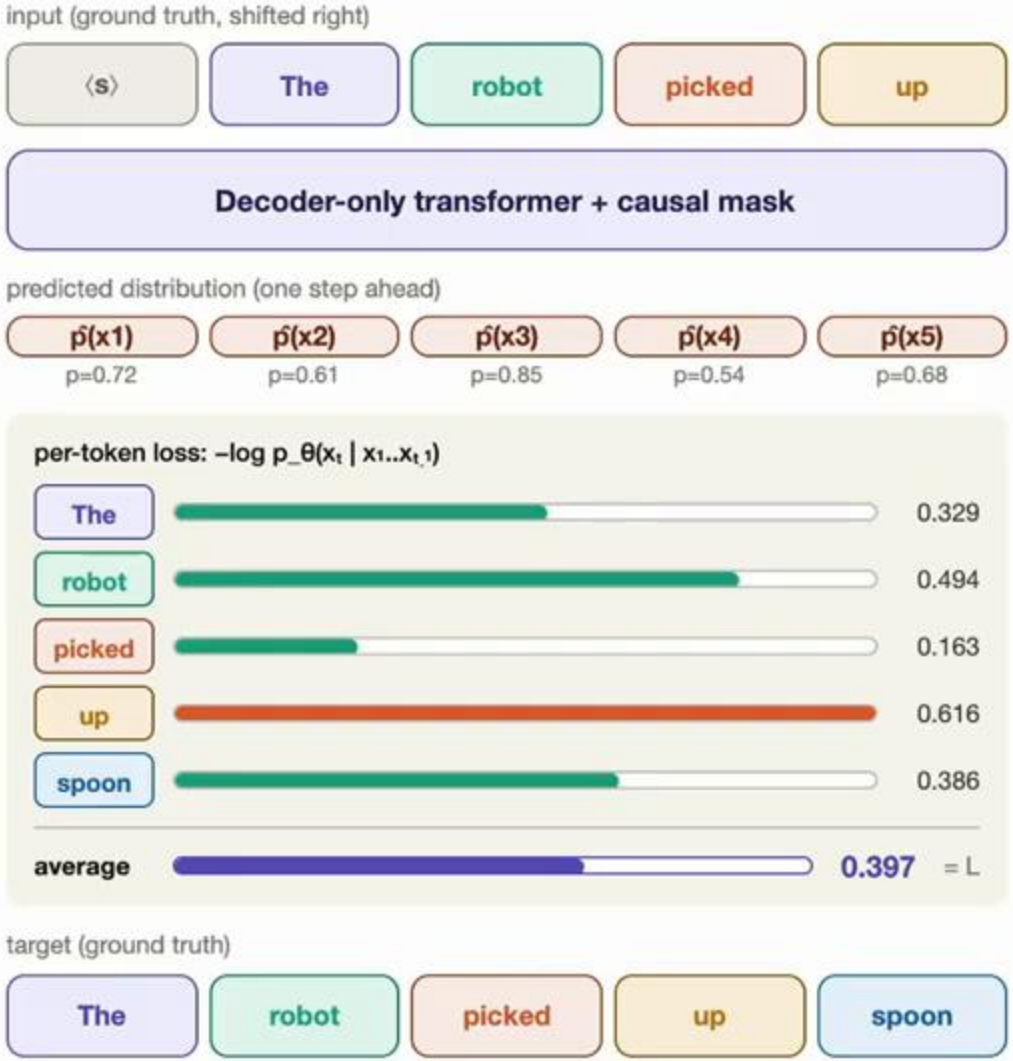
Encoder
Bidirectional self-attention
Every token attends to every other token, past and future



Decoder
Causal self-attention
Each token only attends to itself and past tokens



Training



Teacher Forcing

maximize the likelihood of the next correct token x_t given the true preceding tokens $x_{1:t-1}$

Cross-Entropy Loss

$$L = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

✅ 1x forward pass computes all T losses with Transformers

❌ T forward passes with RNNs

Language Tokenization

- **Characters**: a=0, b=1, c=2,…
  - Small vocabulary
  - Large number of tokens
- **Words**: cat=0, car=1, dog=2, …
  - Large vocabulary
  - Small number of tokens
  - What about new words?

Ideally we would like something in between

Byte-Pair Encoding

- Tokenize based on groupings of characters, prioritizing by frequency
- Generalizes to novel combinations of characters

function BPE (strings C, number of merges k)
  $V \leftarrow$ all unique characters in C # initial vocabulary is characters
  for $i = 1$ to k do # repeat k times
    $t_L, t_R \leftarrow$ most frequent pair of adjacent tokens in C
    $t_{NEW} \leftarrow t_L + t_R$ # concatenate into new token
    $V \leftarrow V + t_{NEW}$ # add to vocabulary
    replace each (t_L, t_R) in C with t_{NEW} # update corpus
  return vocab V

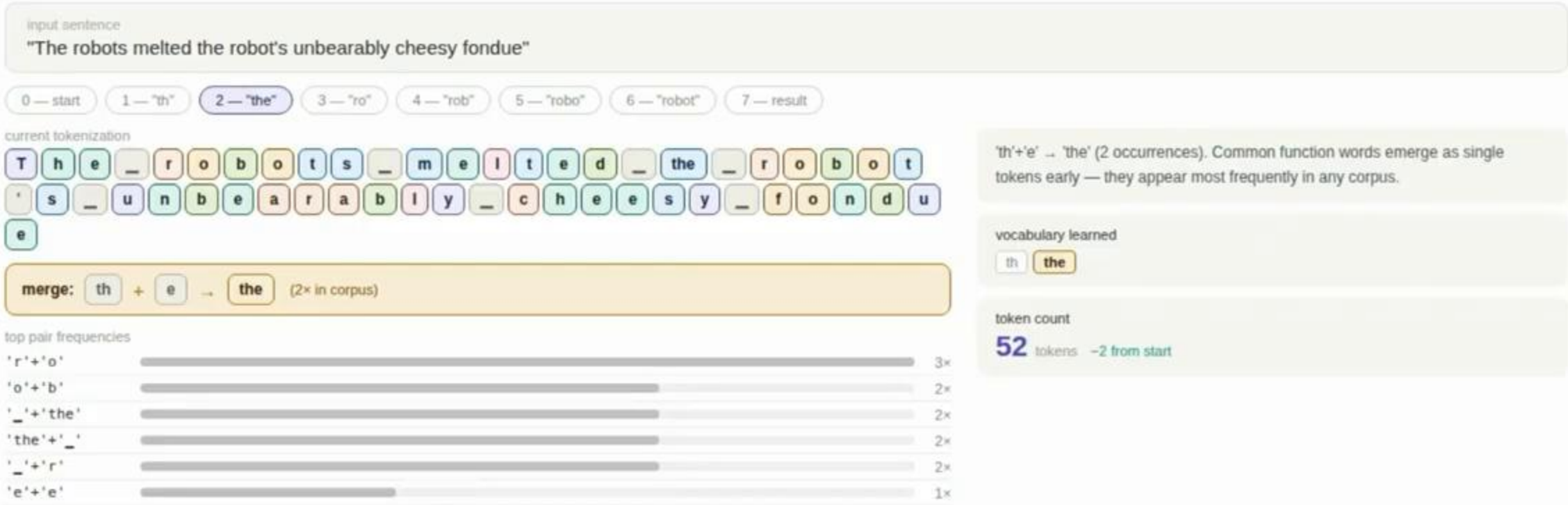
Neural Machine Translation of Rare Words with Subword Units
Sennrich et al., (2015)

BPE Example

- BPE compression factor scales with corpus size

BPE Example

- BPE compression factor scales with corpus size

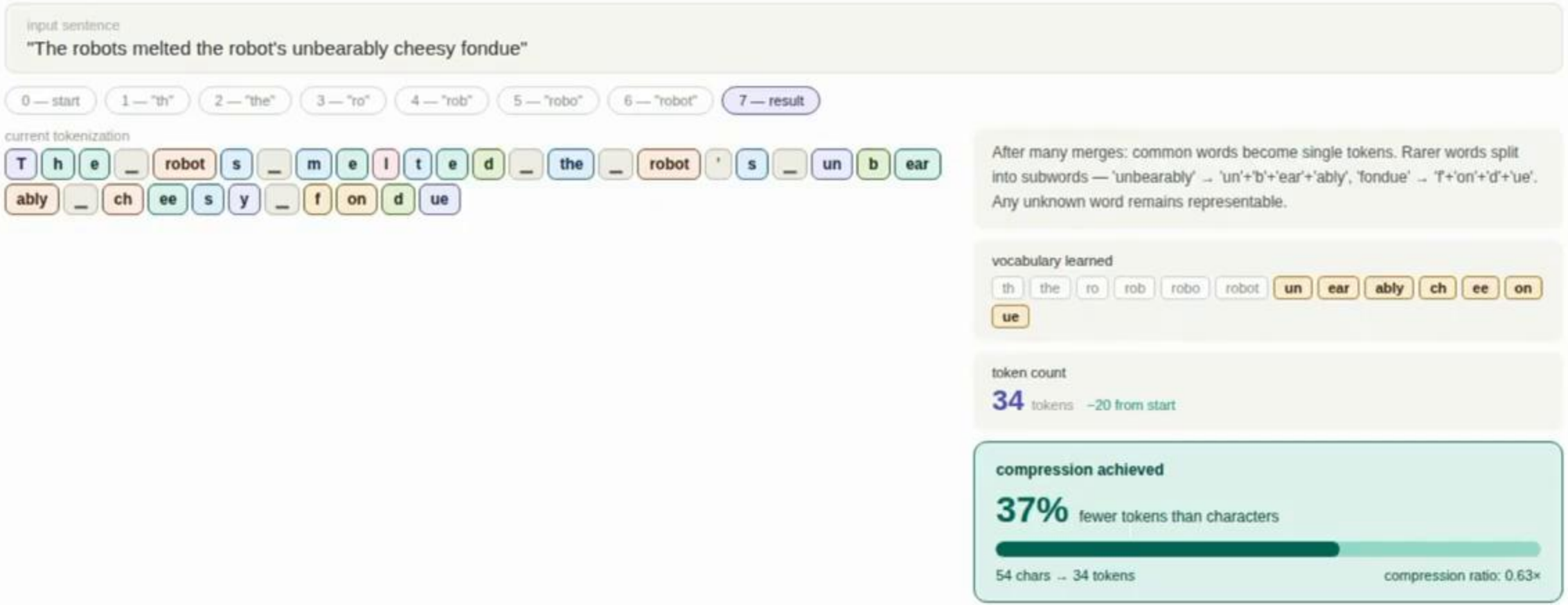


BPE Example

- BPE compression factor scales with corpus size

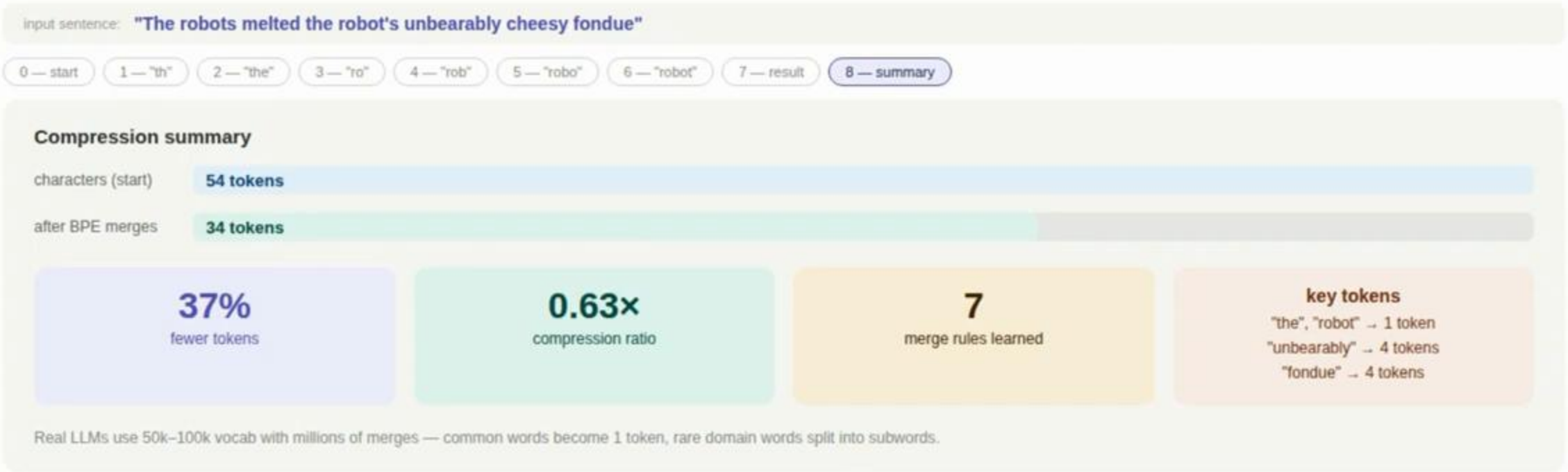
BPE Example

- BPE compression factor scales with corpus size



BPE Example

- BPE compression factor scales with corpus size



Large Language Models

- Decoder-only architectures → simpler to scale
- Extend BPE to work on byte-level → no unknown tokens
- Techniques for scaling:
  - Rotary Position Embeddings (RoPE)
    - Rotates Q and K vectors by an angle proportional to their position, so that $Q \cdot K$ depends only on the relative distance between tokens
  - FlashAttention:
    - Attention with $O(n)$ memory instead of $O(n^2)$ by tiling the computation in fast SRAM, never materializing the full attention matrix

→ RoPE + FlashAttention make training on longer sequences practical

Scaling LLMs

GPT-1

2018

117M params

1B tokens

Pretraining + fine-tuning

GPT-2

2019

1.5B params

10B tokens

Zero-shot generalization

GPT-3

2020

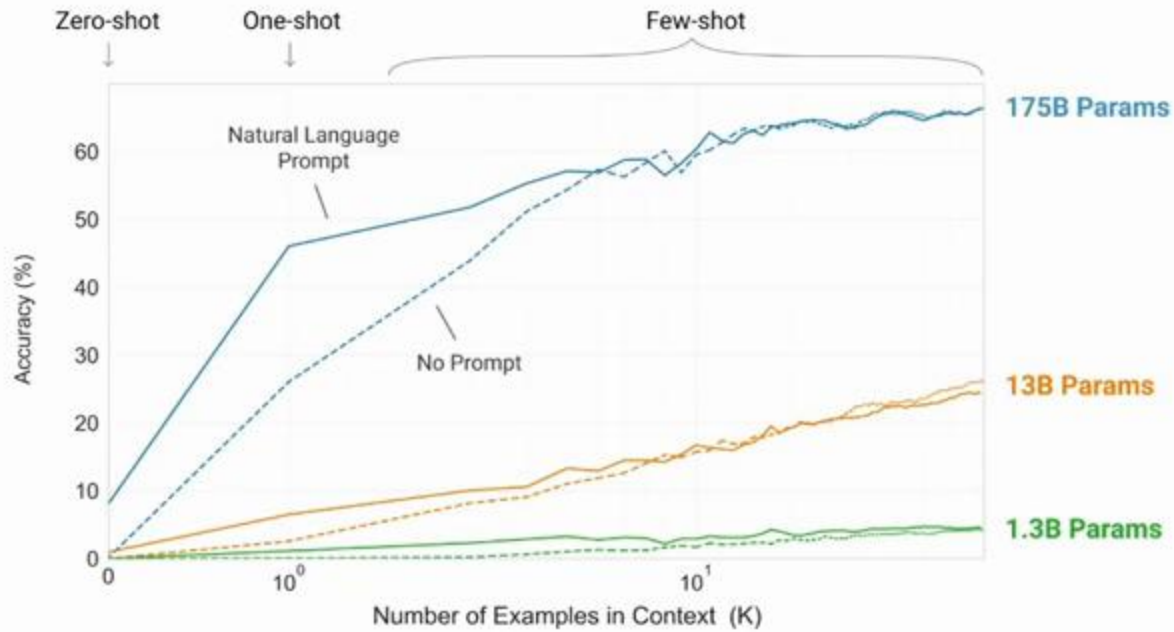
175B params

300B tokens

In-context learning emerges

The Bitter Lesson— Sutton (2019)

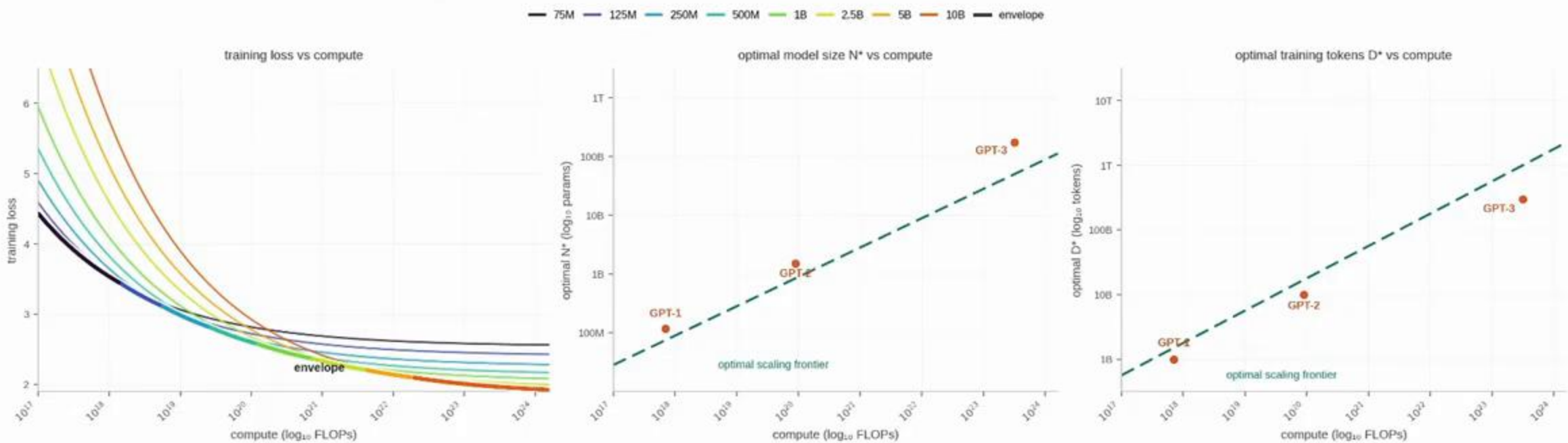
- Methods that scale with compute ultimately beat hand-engineered solutions
  - → In-context learning is an emergent property of scale



Language Models are Few-Shot Learners
Brown et al., OpenAI (2020)

Scaling Laws

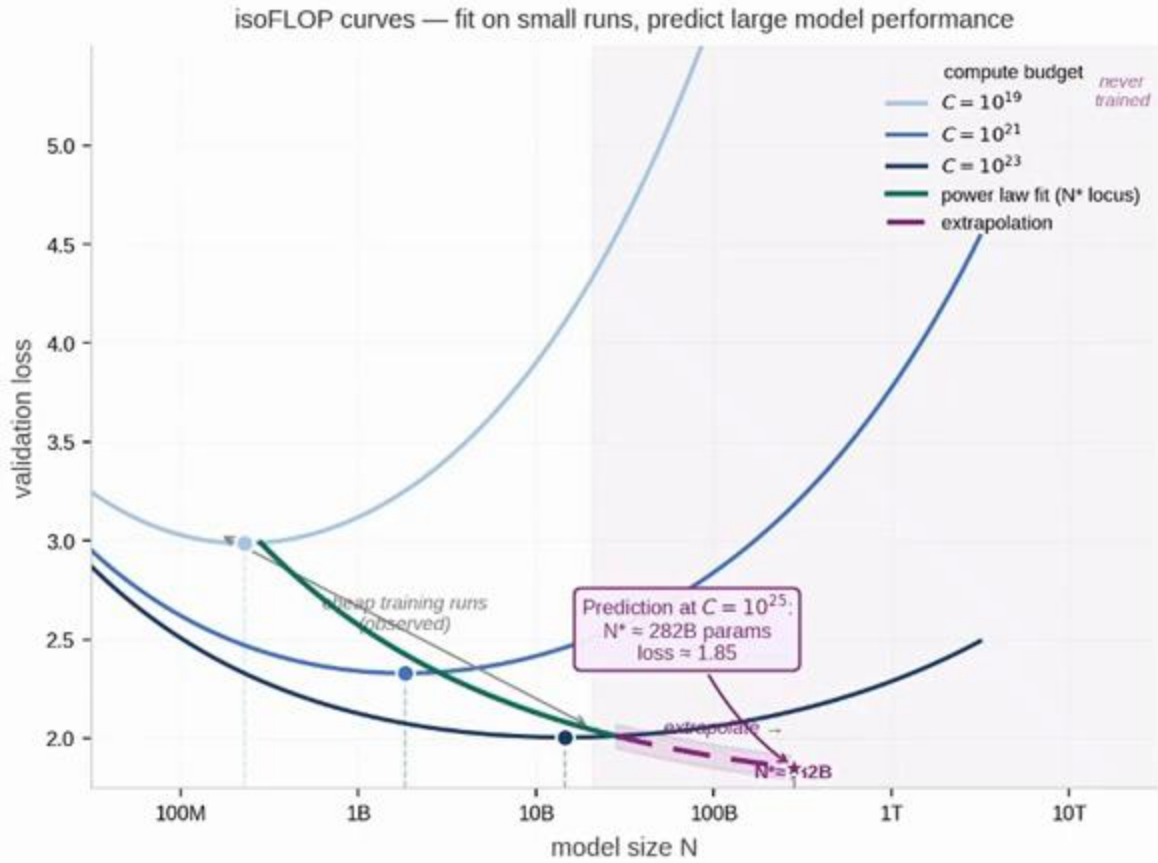
- Given compute budget, scale data & model size equally
  - Optimal model size $N^* \propto C^{0.5}$, data $D^* \propto C^{0.5}$, compute $C \approx 6 \times N \times D$
  - ~20 tokens per parameter for compute-optimality

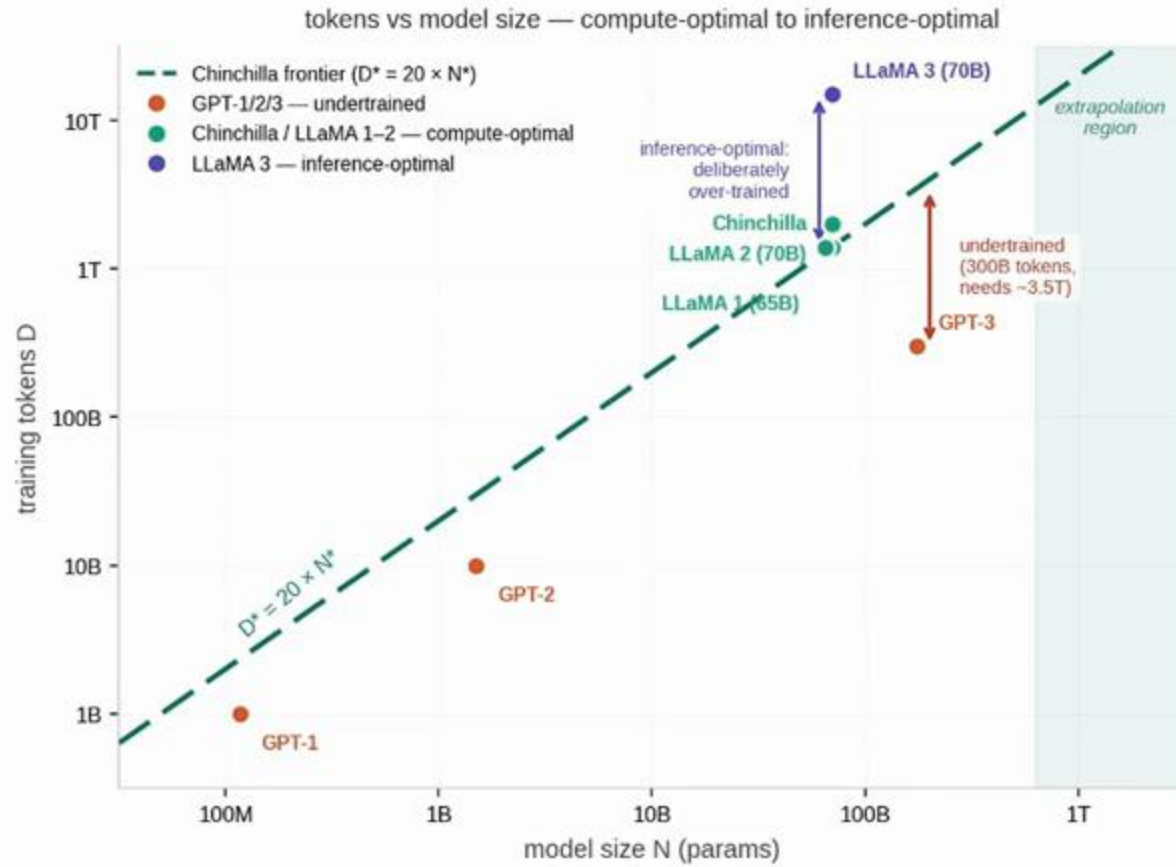


Training Compute-Optimal Large Language Models
Hoffmann et al. (2022)

Scaling Laws: Extrapolating Performance

- Fit power law on cheap runs → extrapolate to predict optimal N and final loss before committing compute

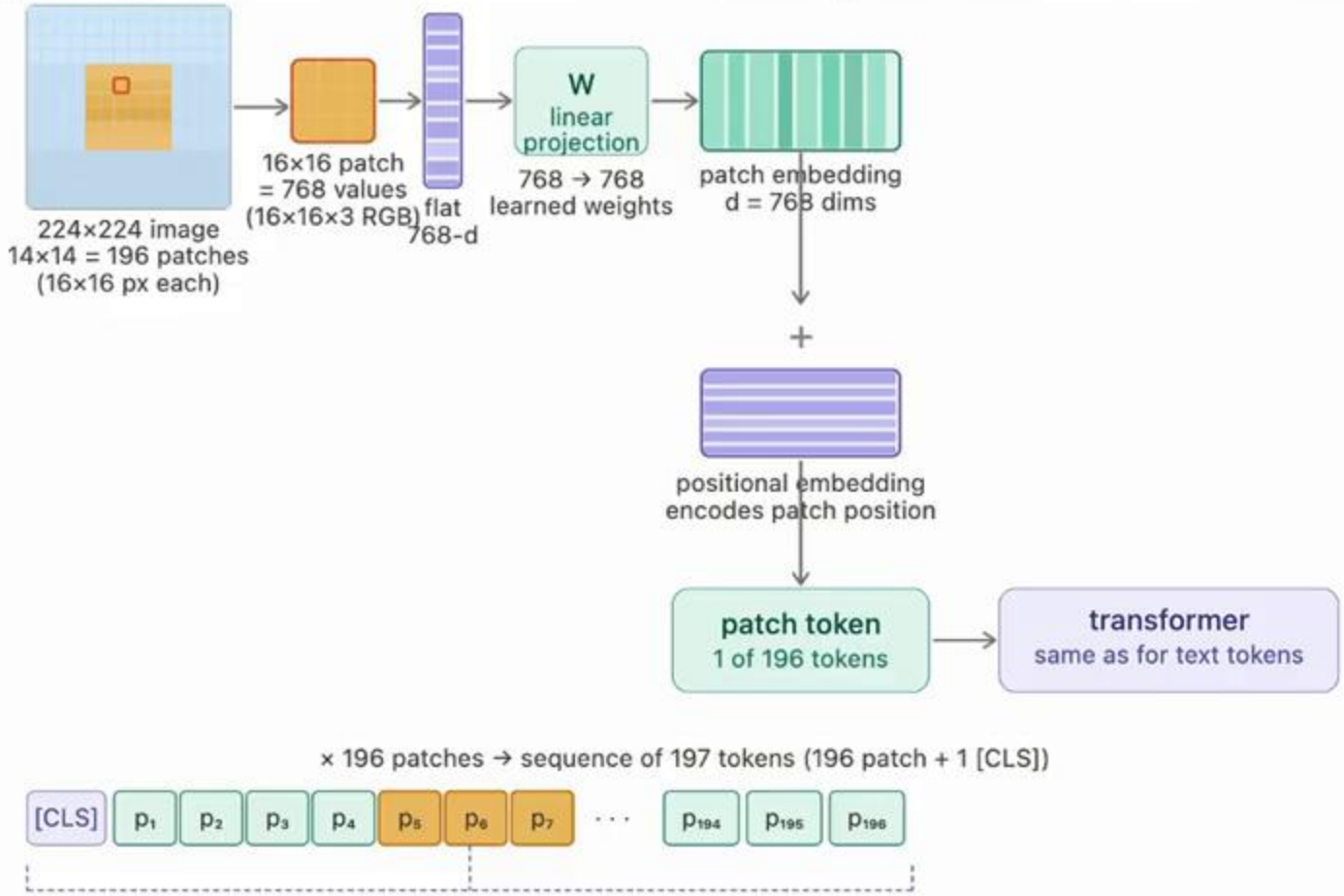




How can we extend LLMs & Transformers to Vision?

Image Tokenization

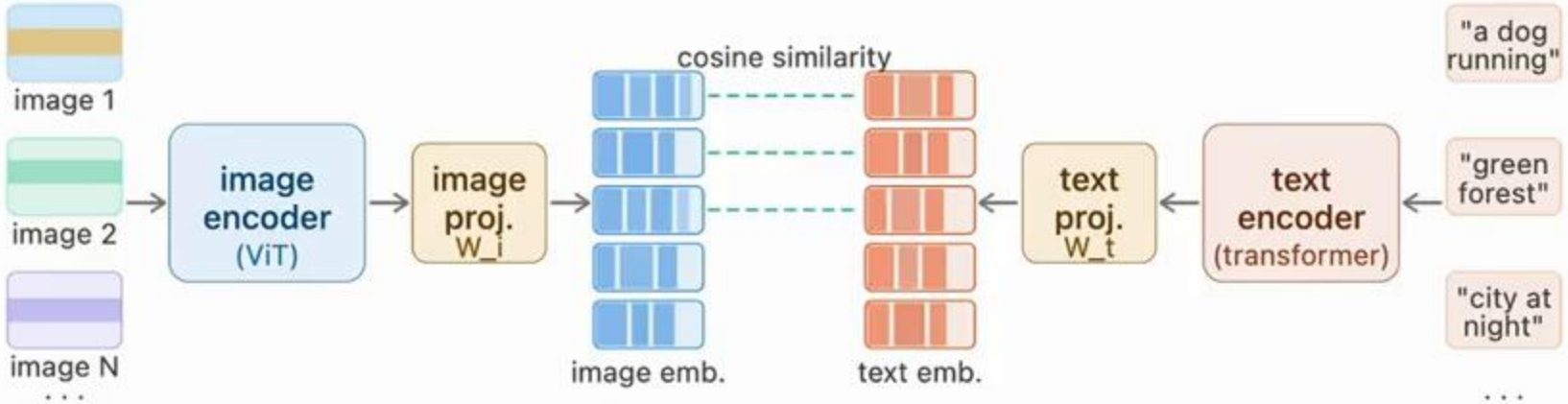
- Split image to patches, flatten & add positional encodings

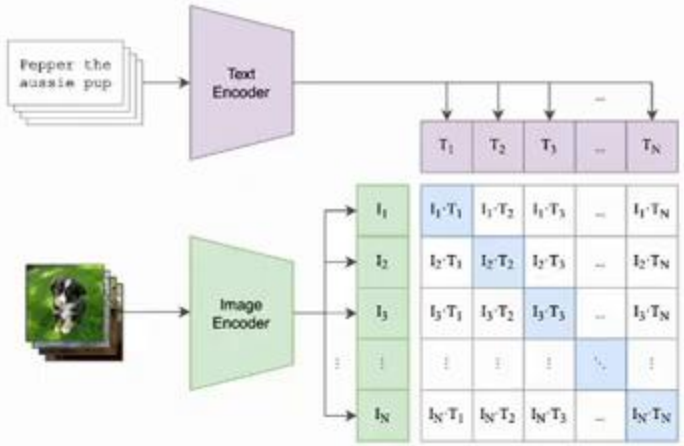


An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale
Dosovitskiy, Beyer et al. (2020)

How does the Transformer know which text tokens correspond to which visual tokens?

CLIP: Two-Tower Contrastive Alignment





Learnable temperature, entropy of the resulting probability distribution

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{2N}\sum_{i=1}^{N}\left[\log \frac{e^{s_{ii}/\tau}}{\sum_{j=1}^{N} e^{s_{ij}/\tau}} + \log \frac{e^{s_{ii}/\tau}}{\sum_{j=1}^{N} e^{s_{ji}/\tau}}\right]$$

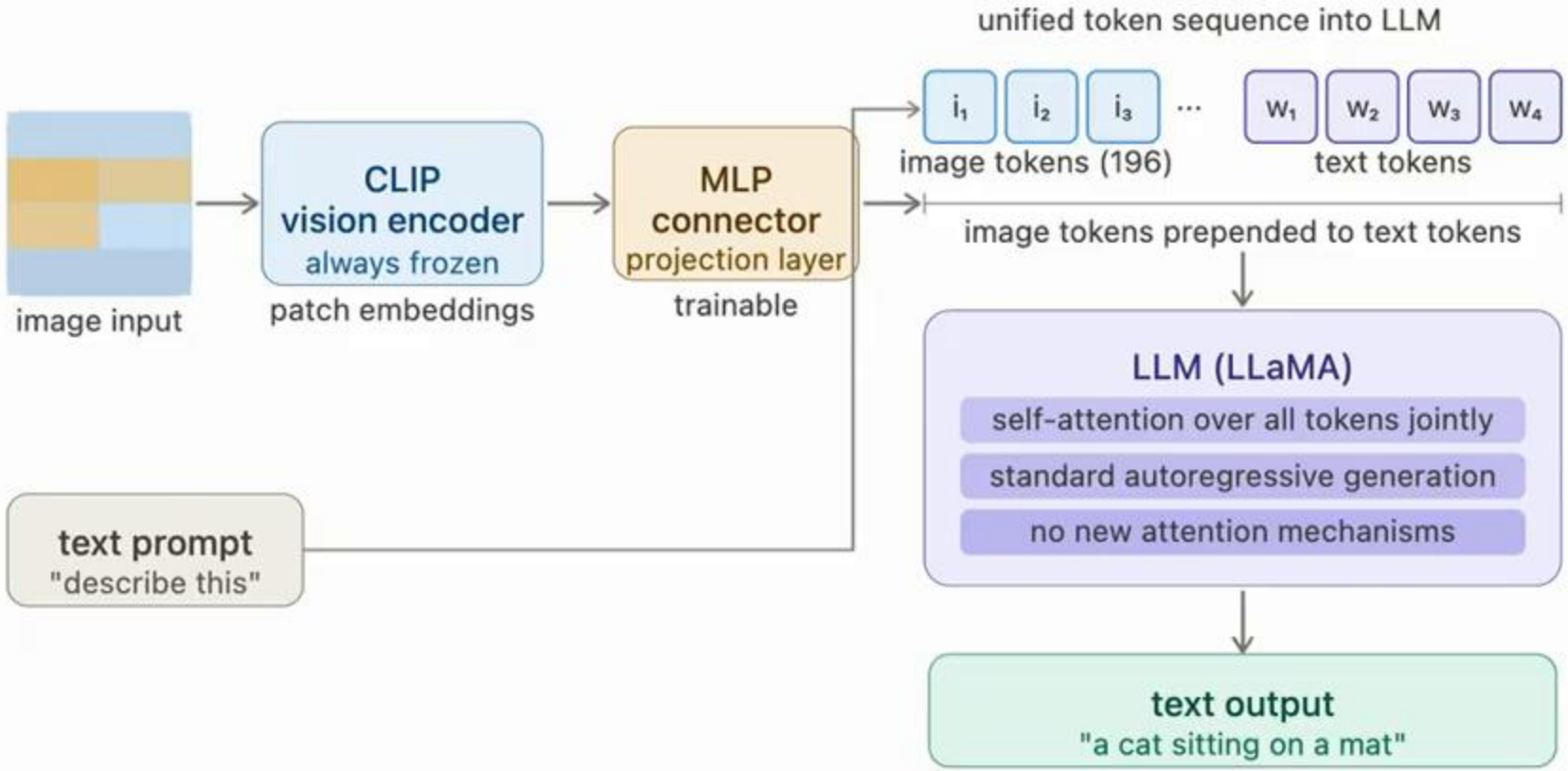
Image-to-Text Loss

Text-to-Image Loss

Learning Transferable Visual Models From Natural Language Supervision
Radford et al. (2021)

Llava: Early-Fusion

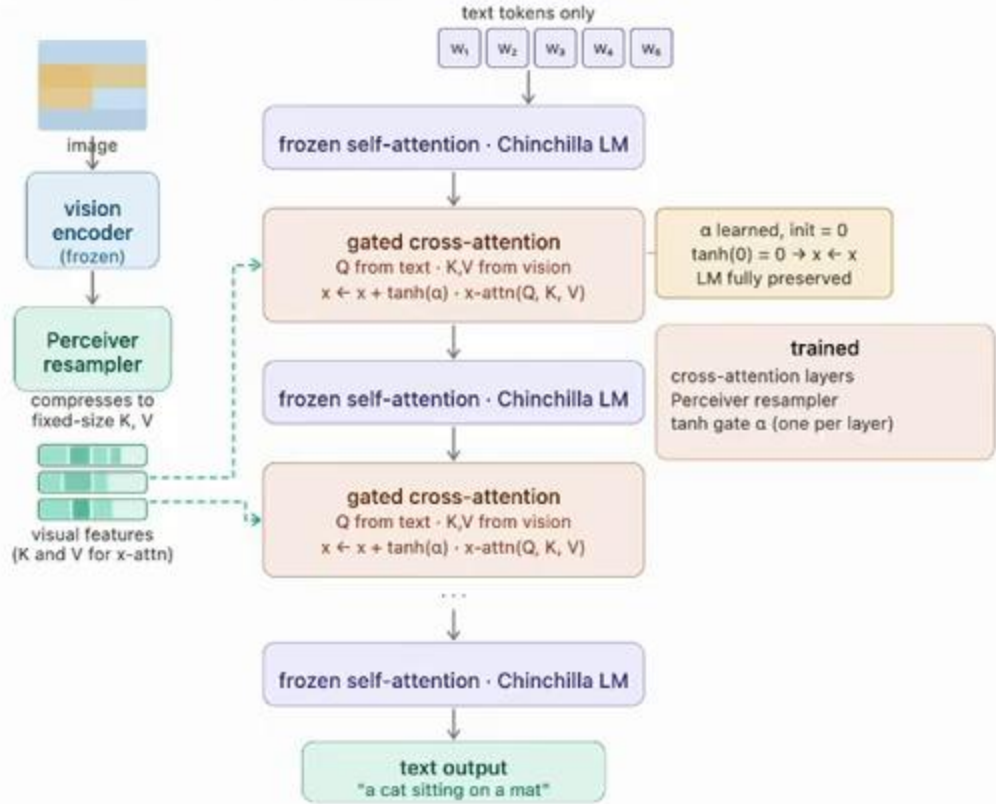
- Prepends image tokens to the text sequence, LLM processes everything in single unified self-attention pass



Visual Instruction Tuning
Liu et al. (2023)

Flamingo: Late-Fusion via Gated Cross-Attention

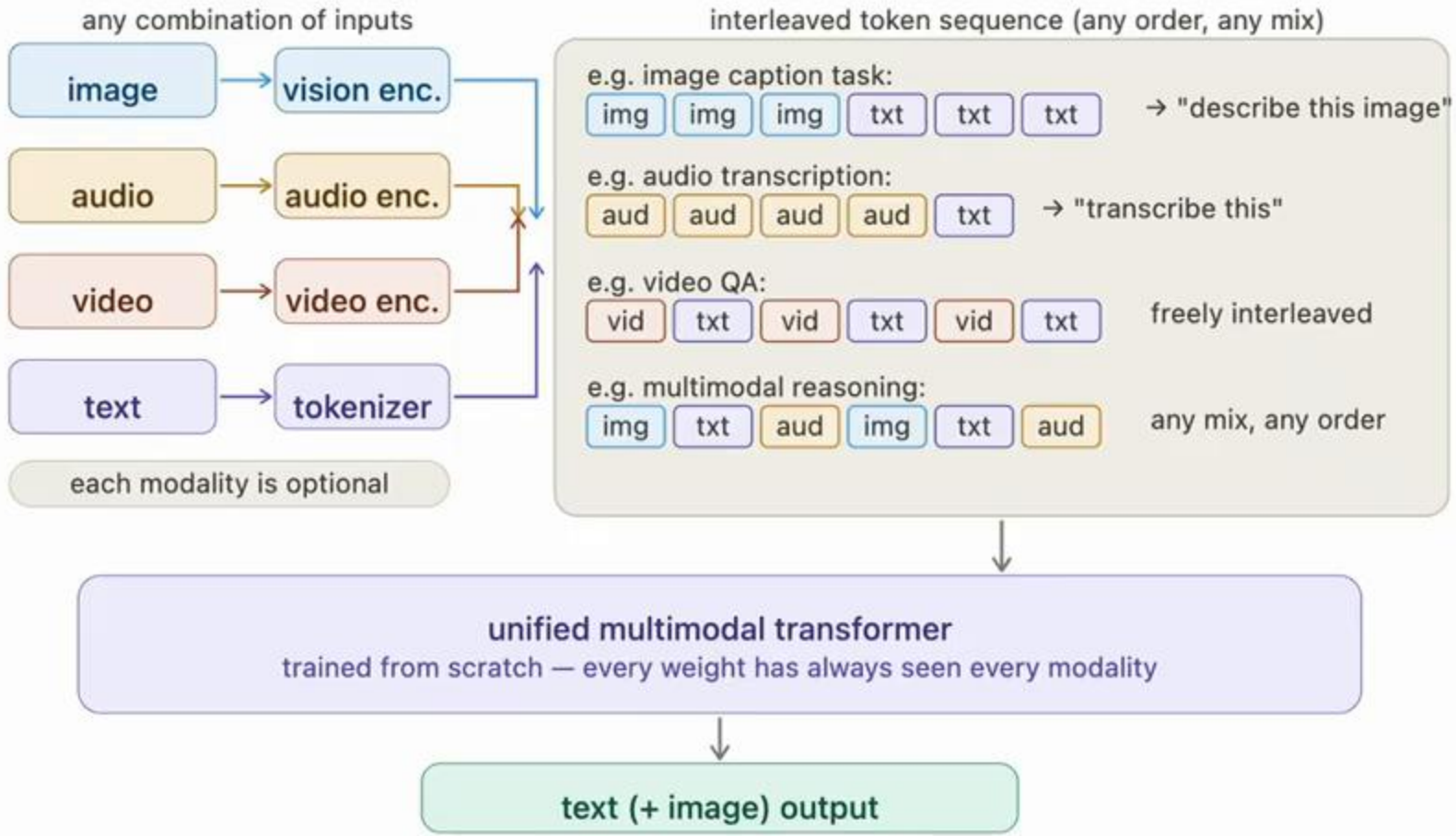
- Vision is injected at every layer via cross-attention, the LM never sees image tokens directly



Flamingo: a Visual Language Model for Few-Shot Learning
Alayrac et al. (2022)

Native Multimodal Models

- Instead of adding vision to an existing LLM, train all modalities jointly from scratch



VLM Taxonomy

early fusion
pre-trained LLM + vision adapter

- **LLaVA · 2023**
  CLIP ViT + MLP + LLaMA
- **PaliGemma 2 · 2024**
  SigLIP + linear proj. + Gemma 2
- **InternVL 2.5 · 2024**
  InternViT-6B + MLP + LLM
- **DeepSeek-VL2 · Dec 2024**
  SigLIP + MLP + DeepSeekMoE
- **Gemma 3 · Mar 2025**
  SigLIP + linear proj. + Gemma 3
- **Phi-4-multimodal · Feb 2025**
  SigLIP-2 + MoE-LoRA + Phi-4
- **Qwen2.5-VL · 2025**
  dyn-res ViT + MLP + Qwen2.5
- **Qwen3-VL · 2025**
  DeepStack + MRoPE + Qwen3
- **Mistral Large 3 · 2025**
  sparse MoE 675B / 41B + vision

late fusion
pre-trained LLM + cross-attention

- **Flamingo · 2022**
  gated cross-attn + Chinchilla LM
- **GPT-4V · 2023**
  undisclosed · likely cross-attn
- **LLaMA 3.2 Vision · 2024**
  cross-attention + LLaMA 3
- **IDEFICS 2 · 2024**
  open Flamingo reproduction
- **NVLM-X · NVIDIA 2024**
  cross-attn + InternViT-6B

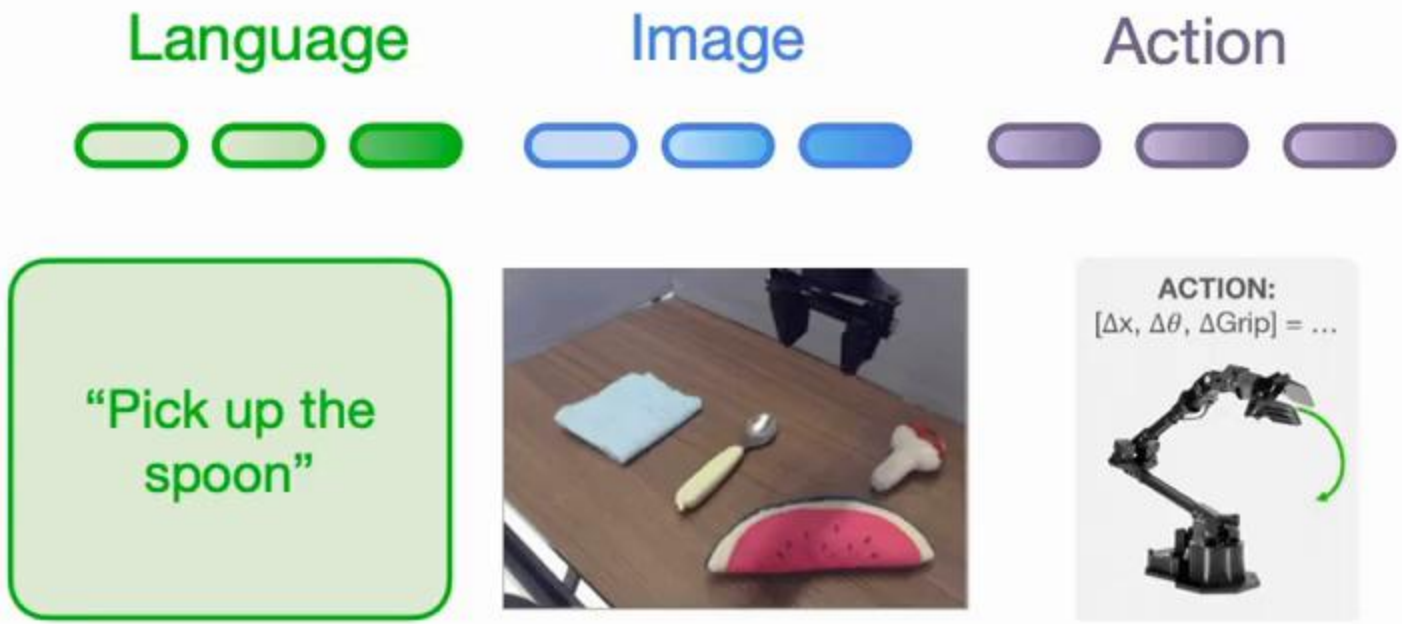
natively multimodal
joint training from scratch

- **Gemini 1 · 2023**
  dense transformer · from scratch
- **Chameleon · 2024**
  VQ-VAE: images as discrete tokens in shared vocabulary
- **GPT-4o · 2024**
  natively multimodal · undisclosed
- **Gemini 3 · 2025**
  sparse MoE · 1M ctx · Deep Think
- **LLaMA 4 · 2025**
  MoE · 400B / 17B · 1M ctx
- **GPT-5 · Aug 2025**
  native MM · unified routing
- **Qwen3.5 · Feb 2026**
  joint pretraining · text + vision

Transformer → LLMs → VLMs →Native Multimodal models

Can we extend it to robot actions?

Robotics as Multimodal Sequence Modeling



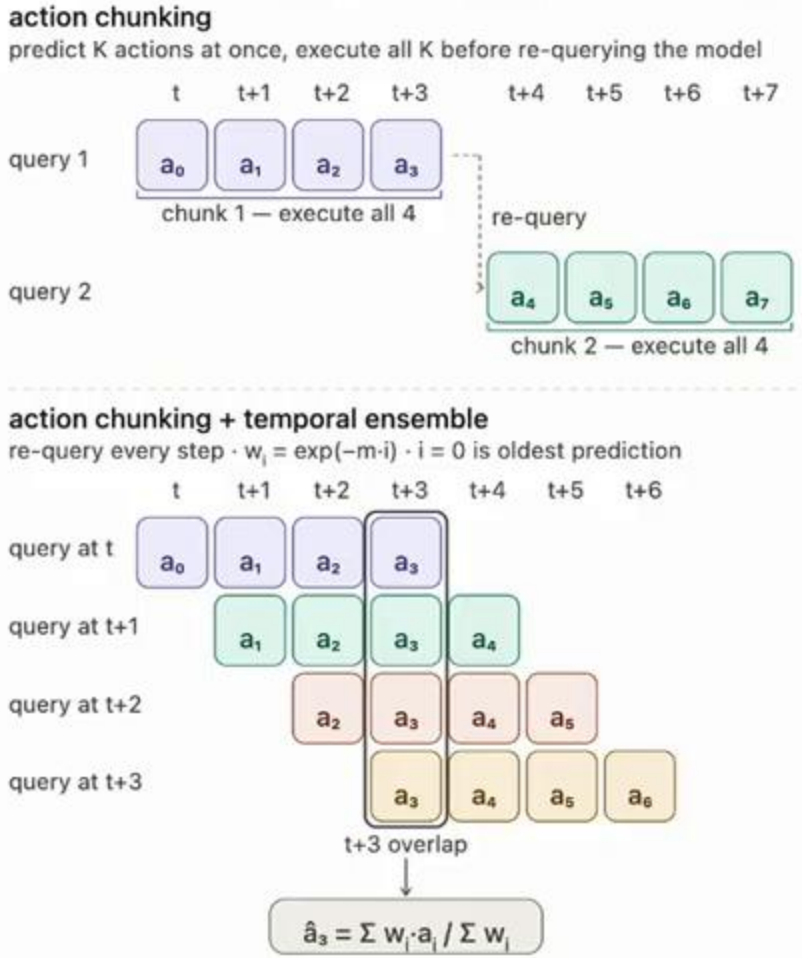
Robot Action Tokenization for VLAs

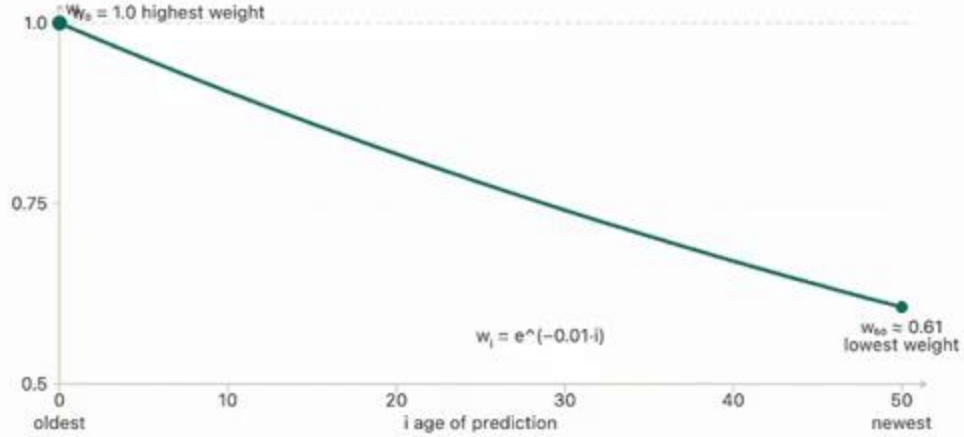
RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control Brohan et al. (2023)

OpenVLA: An Open-Source Vision-Language-Action Model Kim et al. (2024)

Action Chunking

- Predict k future actions for smoother behavior $\pi(a_{t:t+k}|o_t)$





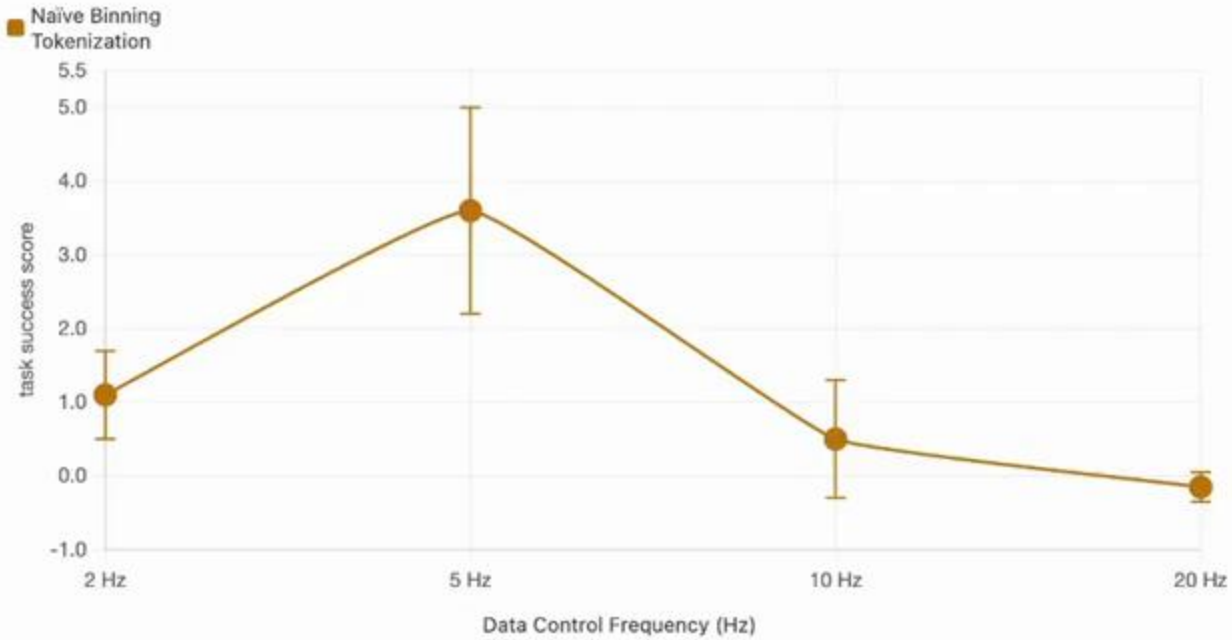
Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware
Zhao et al. (2023)

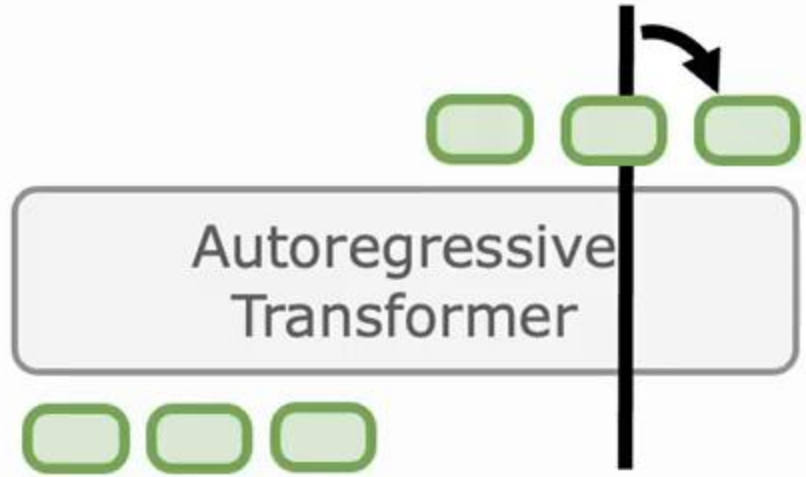
Naïve Action Tokenization

As frequency increases:

- Information per token decreases
- Correlation between token increases

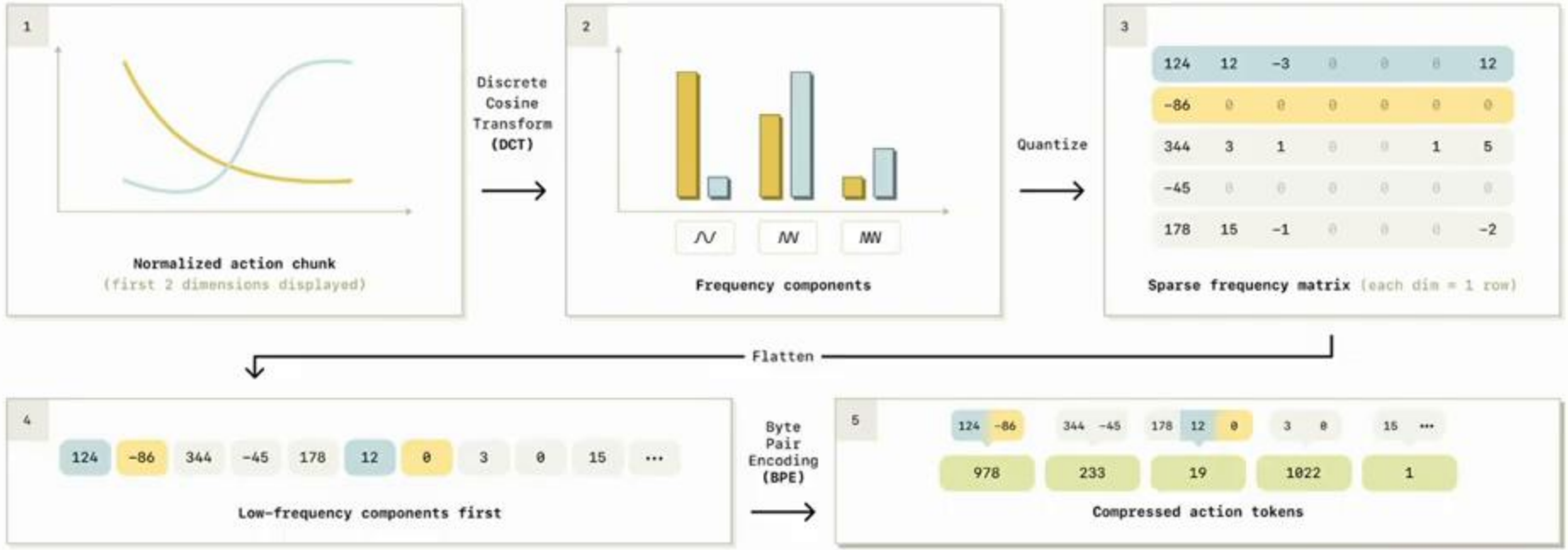
→ Solution: compression





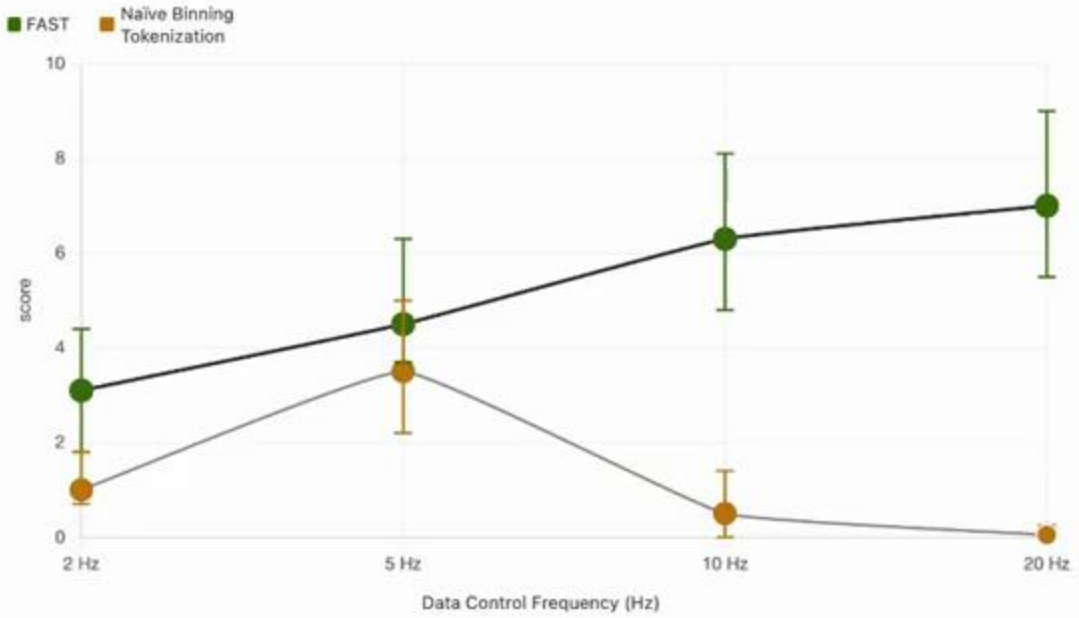
Effective Action Tokenization

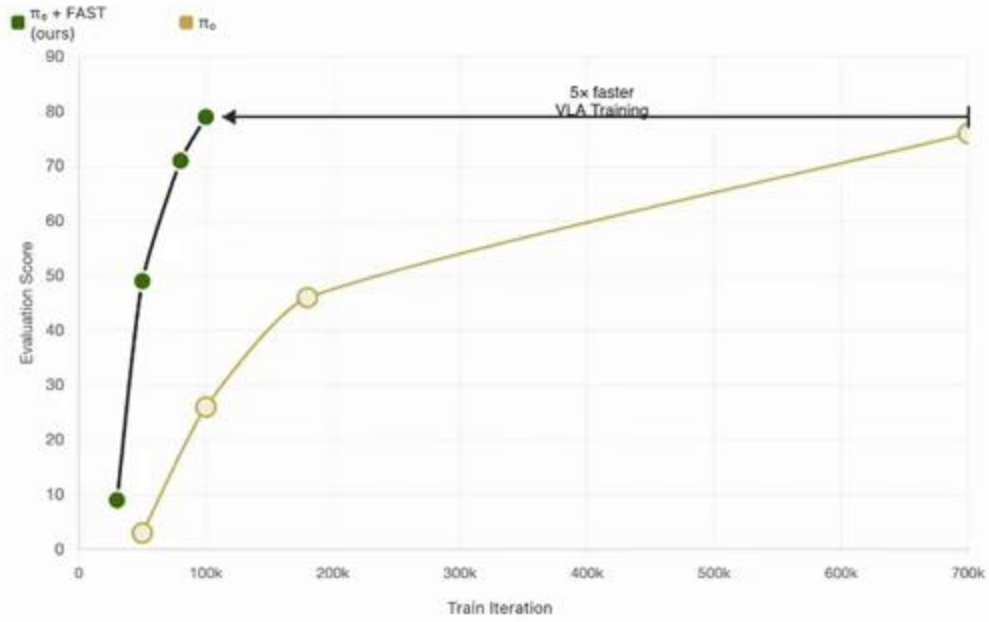
- How to compress continuous robot data?
  - Byte-Pair Encoding (BPE) not suited for continuous data
  - VQ learned compression complex to train
- Key idea: Discrete Cosine Transform (DCT) ≈ JPEG compression



FAST: Efficient Action Tokenizer for VLAs

- Scales to higher frequencies and trains 5x faster





FAST: Efficient Action Tokenization for Vision-Language-Action Models
Pertsch*, Stachowicz*, Ichter, Driess, Nair, Vuong, **Mees** et al. RSS 2025
Finalist Best Conference Paper Award

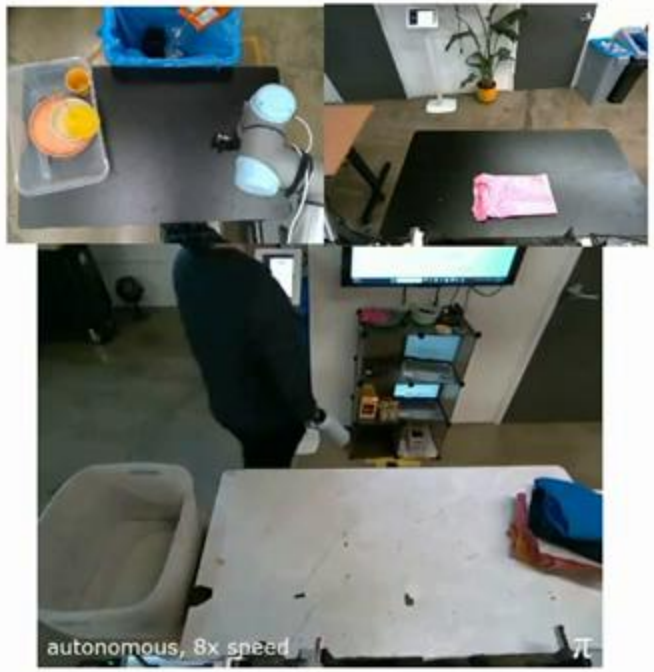
FAST: Efficient Action Tokenizer for VLAs

Generality

@ Berkeley, Stanford, UW

Dexterity

@ Physical Intelligence



FAST: Efficient Action Tokenization for Vision-Language-Action Models

Pertsch*, Stachowicz*, Ichter, Driess, Nair, Vuong, **Mees** et al. RSS 2025

Finalist Best Conference Paper Award

Conclusion

- Robotics is a sequence modeling problem
- Transformers as a scalable architecture → LLMs
- Different architectures for multimodal inputs → VLMs, Native Multimodal models
- Naïve robot tokenization breaks at high frequency control
- Compression-based techniques like FAST popular

Thank you for your attention